

ADVANCED MODEL CONTEXT PROTOCOL AND FASTMCP





LOGISTICS



Class Hours:

- Instructor will set class start and end times.
- There will be regular breaks in class.



Telecommunication:

- Turn off or set electronic devices to silent (not vibrate)
- Reading or attending to devices can be distracting to other students
- Try to delay until breaks or after class

Miscellaneous:

- Courseware
- Bathroom
- Fire drills

DONIS MARSHALL

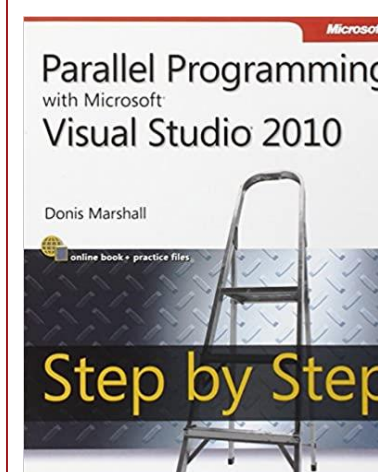
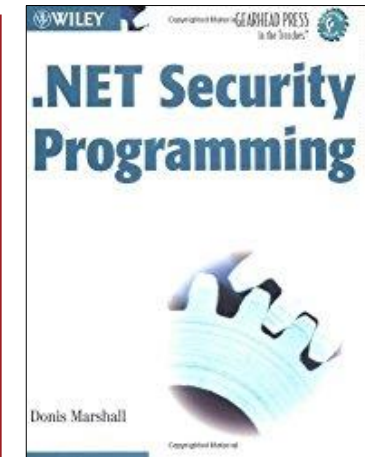
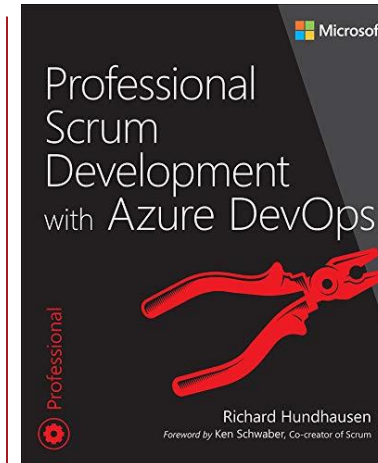
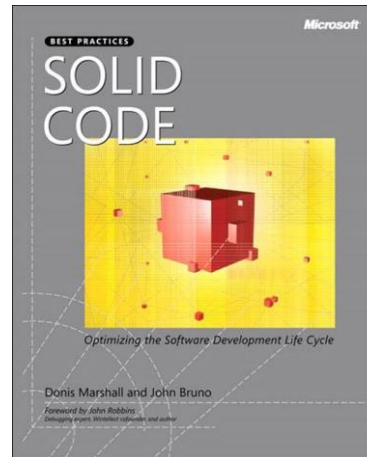
Security Professional

Microsoft MVP

AI Practitioner

Microsoft Certified

Author



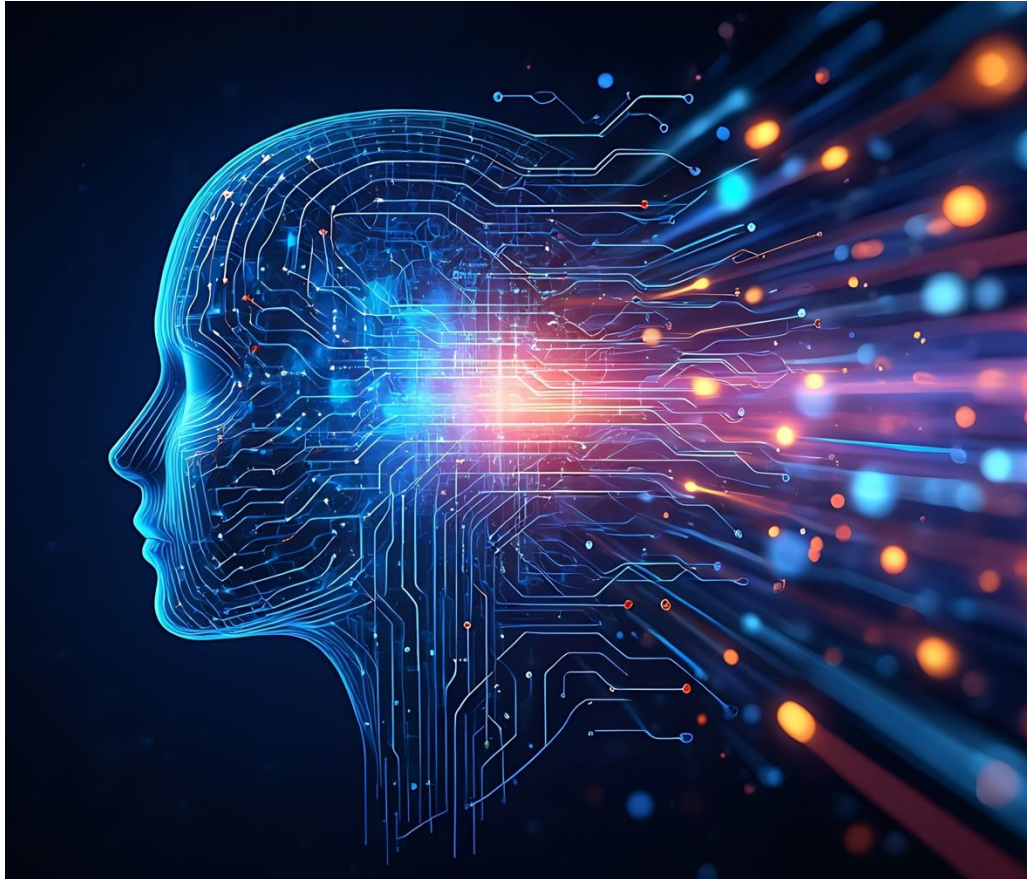
INTRODUCE YOURSELF

Time to introduce yourself:

- Name
- What is your role in the organization
- Indicate GitHub Copilot and MCP Server experience

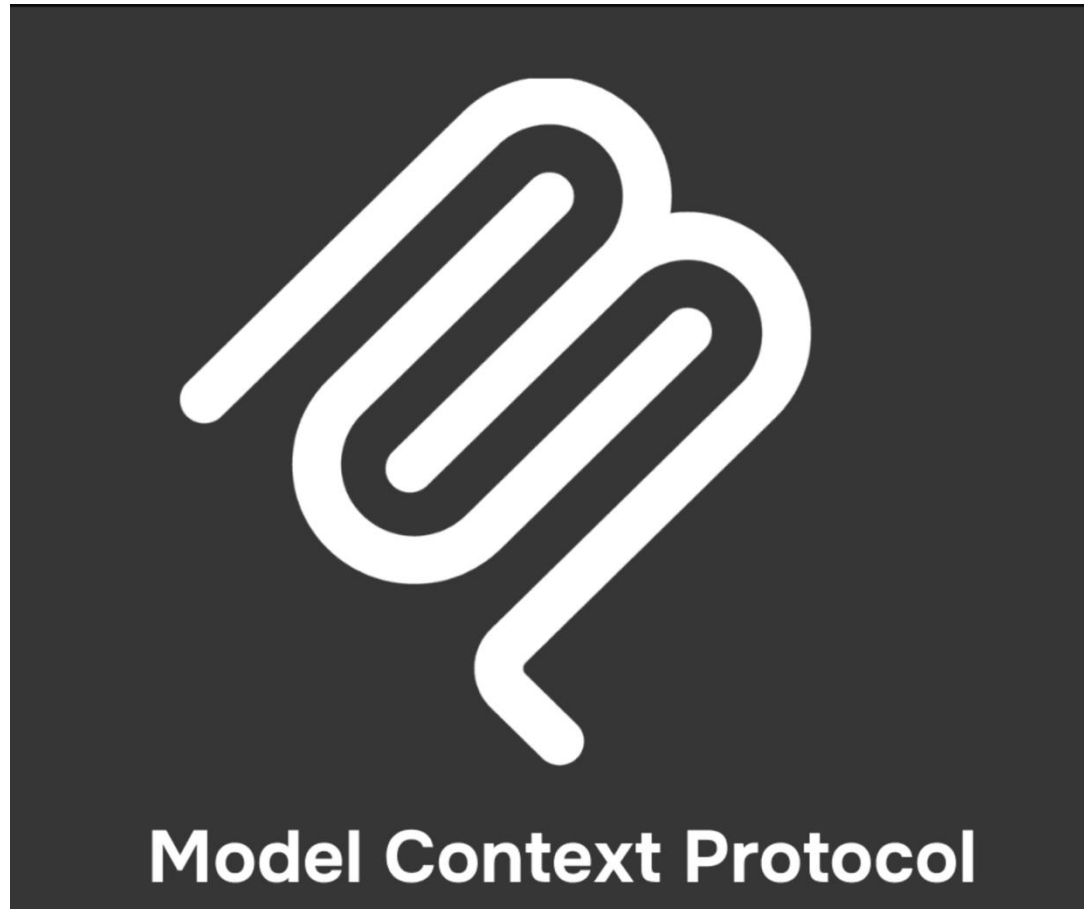


CUSTOM CONTEXT



Large language models are trained on static datasets with a fixed knowledge cutoff, which means they have no awareness of your specific codebase, your organization's internal documentation, or events that occurred after training ended. Custom context solves this by injecting relevant, real-world information directly into the model's active conversation window at query time.

MCP



Model Context Protocol (MCP) is the solution to the static context problem. By standardizing how external sources expose data and actions, MCP lets an AI assistant pull in that additional context on demand (a ticket from a project tracker, the contents of a document, current records from a database) and act on it, all through one consistent interface instead of a patchwork of custom connectors for every source.

CONVERSATION

MCP works like a conversation with clear back-and-forth turns: the client (the AI application) sends a request asking for something—like "run this tool" or "give me this piece of data"—and the server sends back a response with the result.

This request-response pattern keeps things predictable: nothing happens until it's asked for, and every request gets a matching answer, so the AI always knows what it asked for and what it got back.

This makes it easy to plug new tools and data sources into an AI system without confusion about who's supposed to do what next.

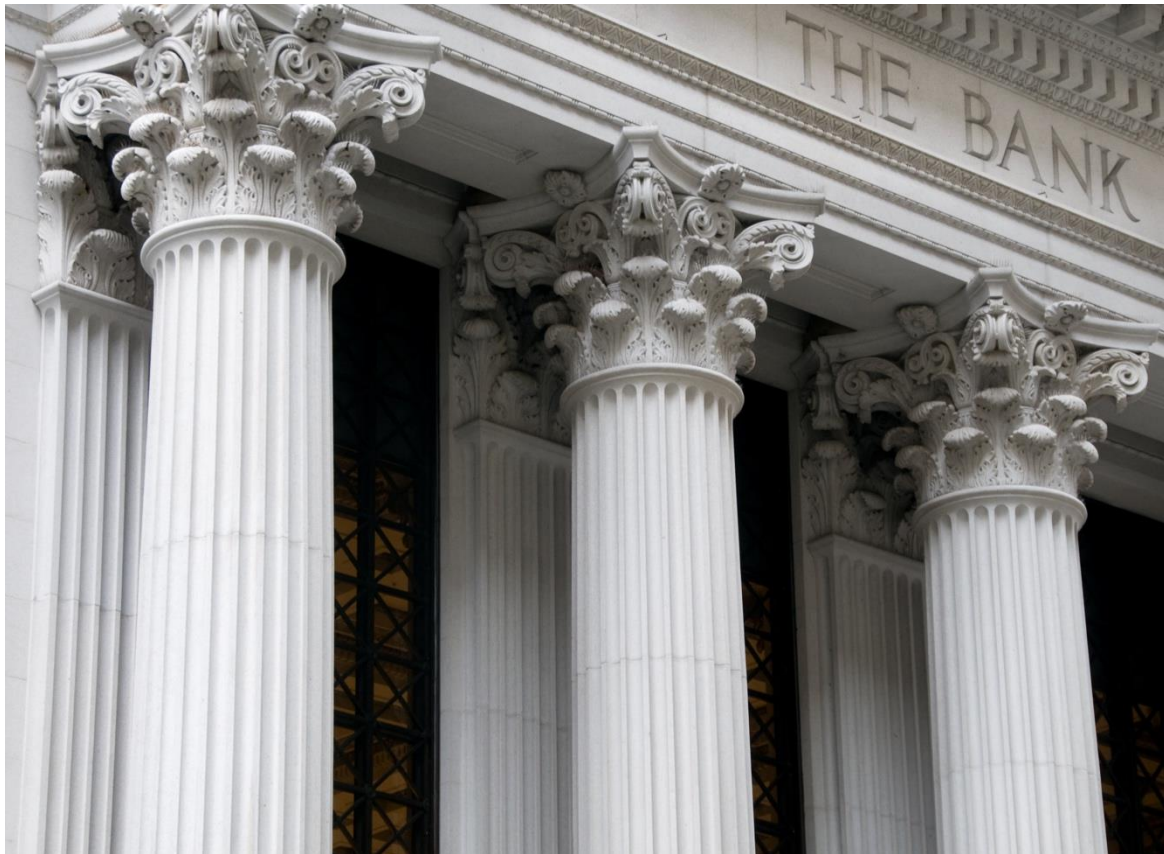
REQUEST / RESPONSE



At a technical level, MCP is similar to how websites work: your browser requests a web server for something, and the server sends back a response.

MCP works the same way, but instead of a browser, it's the AI assistant making the request (like "run this tool" or "get me this data"), and instead of a website's server, it's the MCP server answering back with the result. The AI then uses that answer to keep going with whatever it's doing.

EXAMPLE - GENERAL BANK INFORMATION



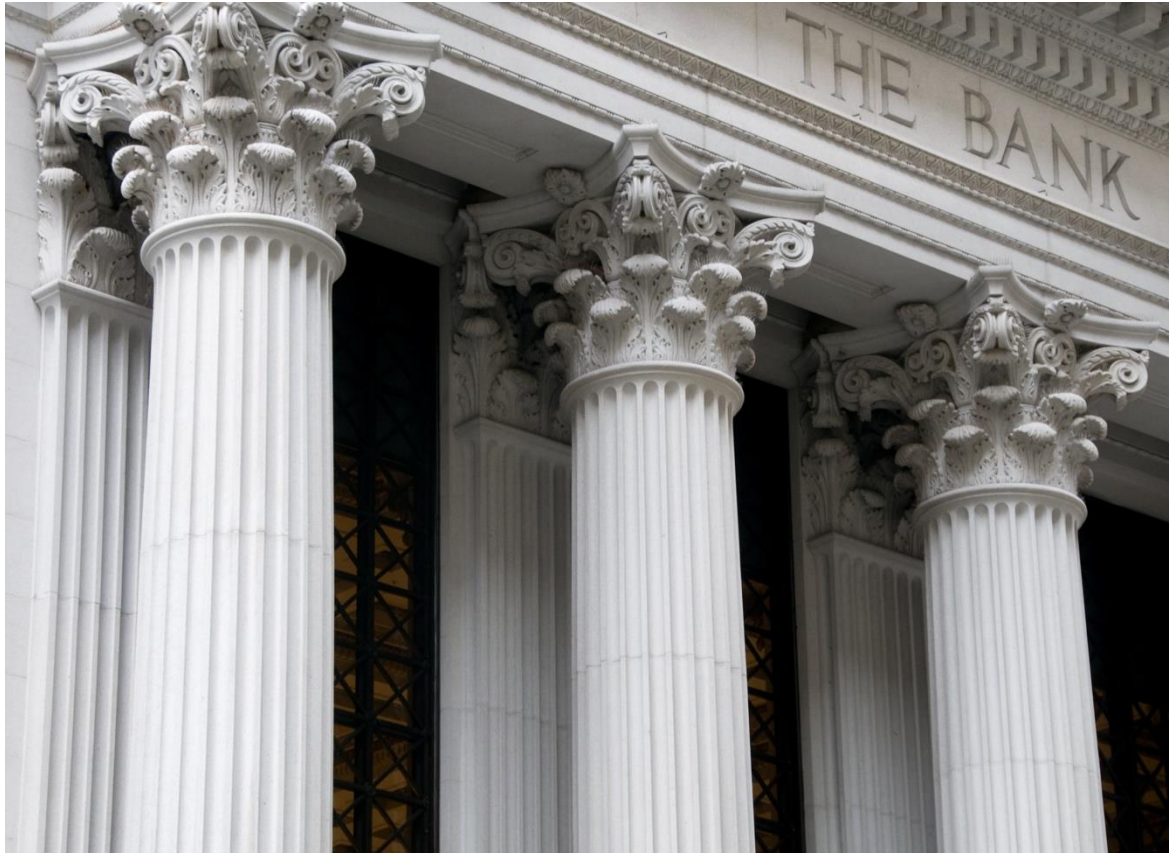
An MCP server can connect a bank chatbot to approved information sources. Customers can ask questions such as “Where is the nearest branch?”, “What are today’s hours?”, or “Where is the closest ATM?” The MCP server retrieves accurate location, hours, and service information directly from the bank’s systems.

EXAMPLE - GENERAL BANK INFORMATION - 2

The chatbot can also provide public financial information about the bank, including stock price, market capitalization, dividends, investor relations resources, annual reports, and recent news. The MCP server retrieves this information from approved sources, ensuring that customers receive current and reliable data.

In addition, the chatbot can answer questions about bank executives, board members, and company leadership. The MCP server helps ensure that only approved public information is shared, while providing security, auditability, and compliance with the bank's policies and procedures.

EXAMPLE – FINANCIAL QUOTING



An MCP server can provide AI assistants with secure access to a bank's financial quote systems. Instead of generating rates or prices on its own, the AI requests current information from approved bank systems through MCP tools. This ensures that customers and employees receive accurate, up-to-date quotes.

EXAMPLE – FINANCIAL QUOTING - 2

For example, a banker might ask an AI assistant for current rates on certificates of deposit (CDs), Treasury bonds, or money market products. The AI sends the request to the MCP server, which retrieves the latest pricing from the bank's internal systems and returns the results for the AI to summarize and explain.

Using MCP provides security, compliance, and auditability. The bank controls which tools are available, who can access them, and what data can be returned. All requests can be logged and monitored, helping ensure that financial information is delivered consistently and according to regulatory requirements.

POP QUIZ: SCENARIOS

What other scenarios could benefit from a MCP server and naturally the ability to extend the context of an LLM?



10 MINUTES

GITHUB COPILOT



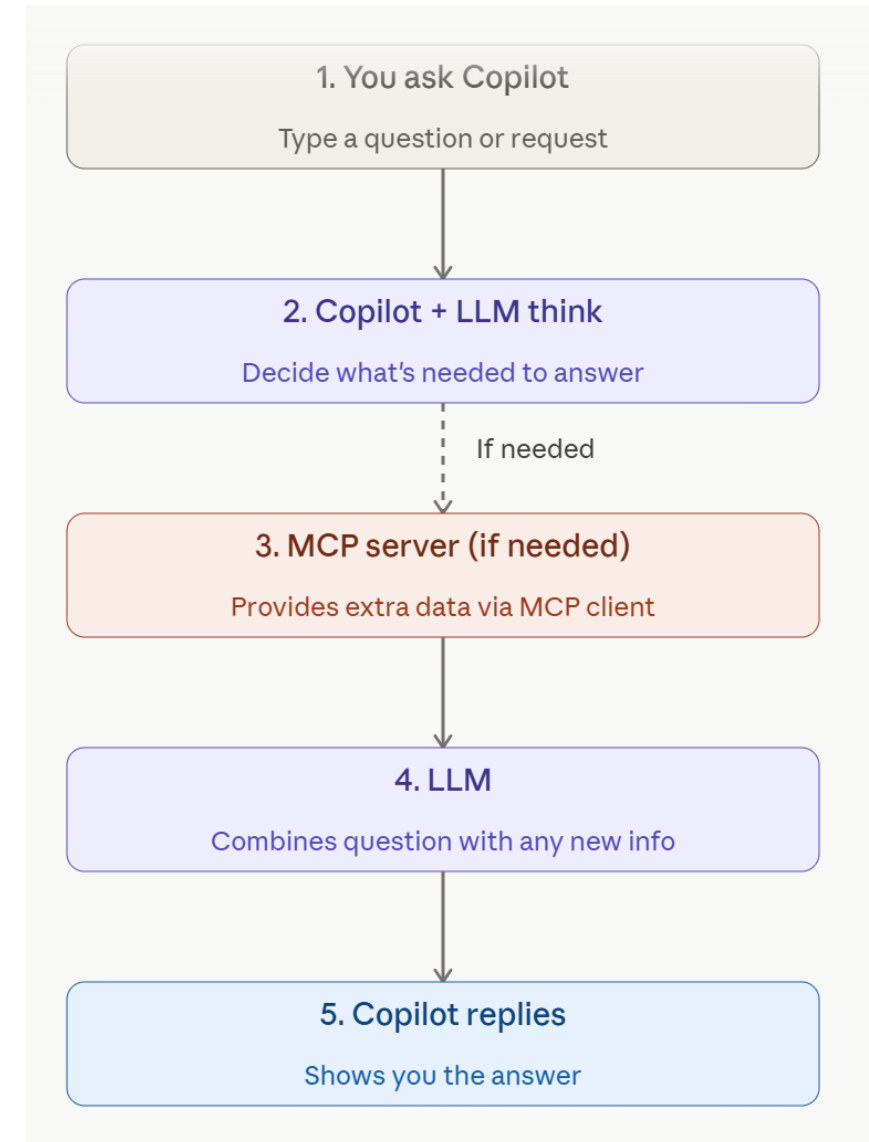
GitHub Copilot

GitHub Copilot is the AI assistant for this class, under the hood it's powered by an LLM—the actual AI model that understands your code and generates suggestions or answers. Copilot offers multi-model choice, with models from Anthropic, Google, and OpenAI available, so the "brain" behind Copilot can vary depending on what you pick.

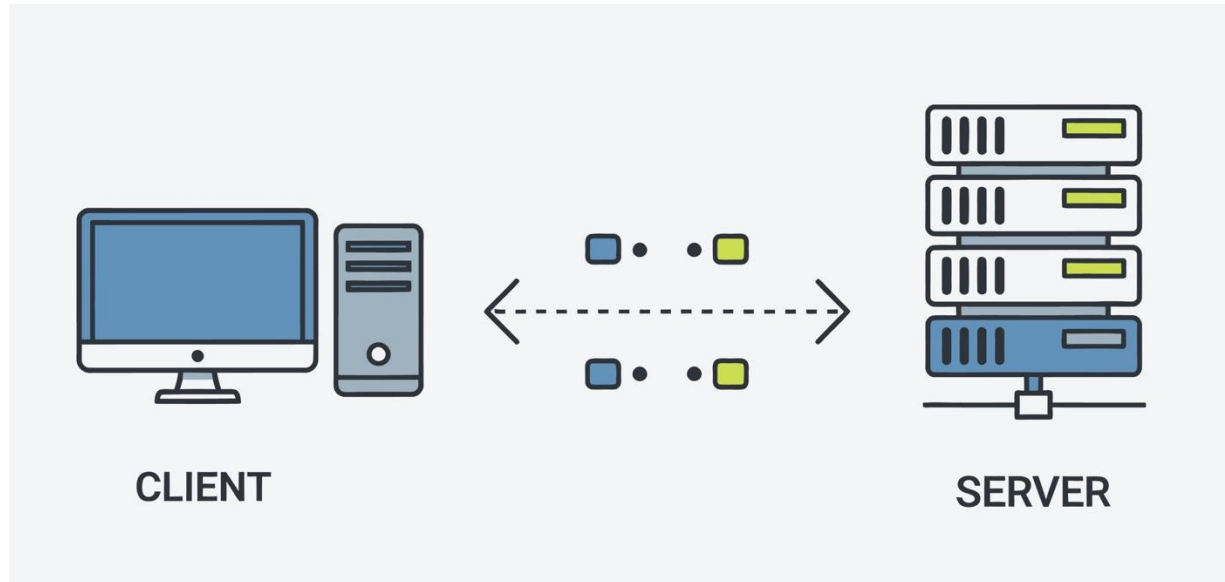
MODEL CONTEXT PROTOCOL (MCP)

GitHub Copilot and the LLM figure out what they need, an MCP server optionally chips in extra data, the LLM folds that into your original question, and Copilot hands back the answer.

The dashed arrow marks the one optional step—if the LLM already has everything it needs, it can skip straight to combining and answering.



CLIENT-SERVER ARCHITECTURE



MCP follows a client-server architecture, MCP servers and MCP clients. The AI application acts as the MCP client, while the MCP server provides access to approved capabilities. For example, an MCP server might allow an AI assistant to query a customer database, retrieve bank branch locations, generate financial quotes, access documentation, or create support tickets.

THE PROTOCOL

The Model Context Protocol (MCP) is an open standard that enables AI applications and external systems to communicate using a standardized set of JSON-RPC APIs. Every MCP session begins with an initialize request, which serves as a handshake between the client and server. During this exchange, the MCP client and MCP server negotiate the MCP protocol version, advertise their capabilities, and exchange metadata such as names and versions.

This initialization step ensures that both sides understand what features are supported before any tools, resources, or prompts are accessed.

THE PROTOCOL - 2

After initialization, the client can discover and interact with the server's capabilities using standard MCP APIs. Common discovery APIs include `tools/list`, `resources/list`, and `prompts/list`, which allow the client to determine what functionality is available. The client can then invoke a tool using `tools/call`, retrieve information using `resources/read`, or obtain a reusable prompt template using `prompts/get`.

Because these APIs are standardized, an MCP client such as ChatGPT, Claude, VS Code, or a custom application can interact with any compliant MCP server without requiring a custom integration.

SUPPORTING APIS



MCP also defines a set of supporting APIs for operational and user experience features. These include ping for connectivity testing, logging/setLevel for controlling logging behavior, completion/complete for intelligent completions, and notification APIs for progress reporting and capability changes. Most developers do not implement these APIs directly.

FASTMCP

FASTMCP

FastMCP is a tool that makes it much easier to build MCP connections in Python.

FastMCP is a tool that makes it much easier to build MCP connections in Python. Normally, hooking an AI tool up to MCP means writing a lot of technical, repetitive code to handle messaging formats and connection details.

FASTMCP - 2

FASTMCP

FastMCP handles all that boring stuff automatically—you just write a normal Python function, slap a simple label on it (like `@mcp.tool`), and it's ready to go.

This means developers can build things faster with way less code, and since FastMCP has already ironed out the tricky bugs, what you build tends to work more reliably too. That's part of why it's become the go-to choice for most MCP projects today.

FASTMCP - 3

FASTMCP

FastMCP 1.0 was actually incorporated into the official MCP Python SDK in 2024, and the standalone project is now downloaded a million times a day, with some version of FastMCP powering an estimated 70% of MCP servers across all languages.

For this reason, when people say "FastMCP," they are mostly referring to the Python-specific high-level wrapper, but the *name* and pattern have also been ported to other ecosystems (more below).

FASTMCP – LANGUAGE SUPPORT

The logo for FASTMCP, featuring the word "FASTMCP" in a bold, italicized, black sans-serif font.

The official SDK ecosystem now covers seven languages:

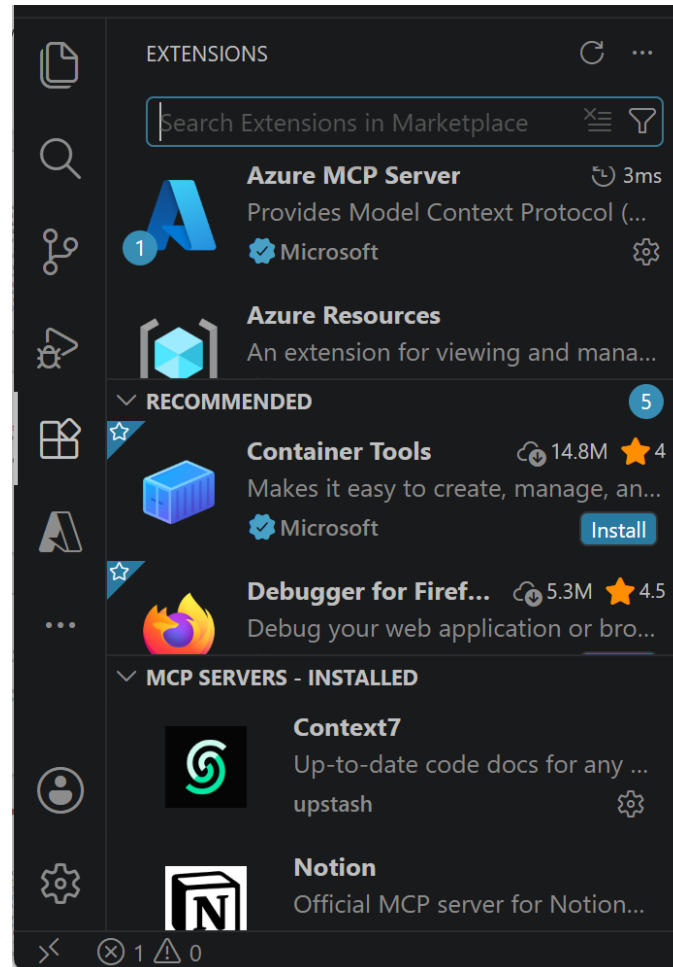
- Python
- TypeScript
- Golang
- Java
- Kotlin
- C#
- Rust

JAVA - MCP

For Java specifically, there are two layers of support:

- The official Java SDK (modelcontextprotocol/java-sdk) — maintained in collaboration with Spring AI, providing client and server primitives, annotation-based method handling, and OAuth 2.0 / API-key security support.
- Spring AI MCP, which extends the Java SDK with Spring Boot starters for both client and server — this is the layer most enterprise Java teams would actually touch, similar in spirit to how FastMCP sits on top of the raw Python SDK.

MCP SERVERS: EXTENSIONS

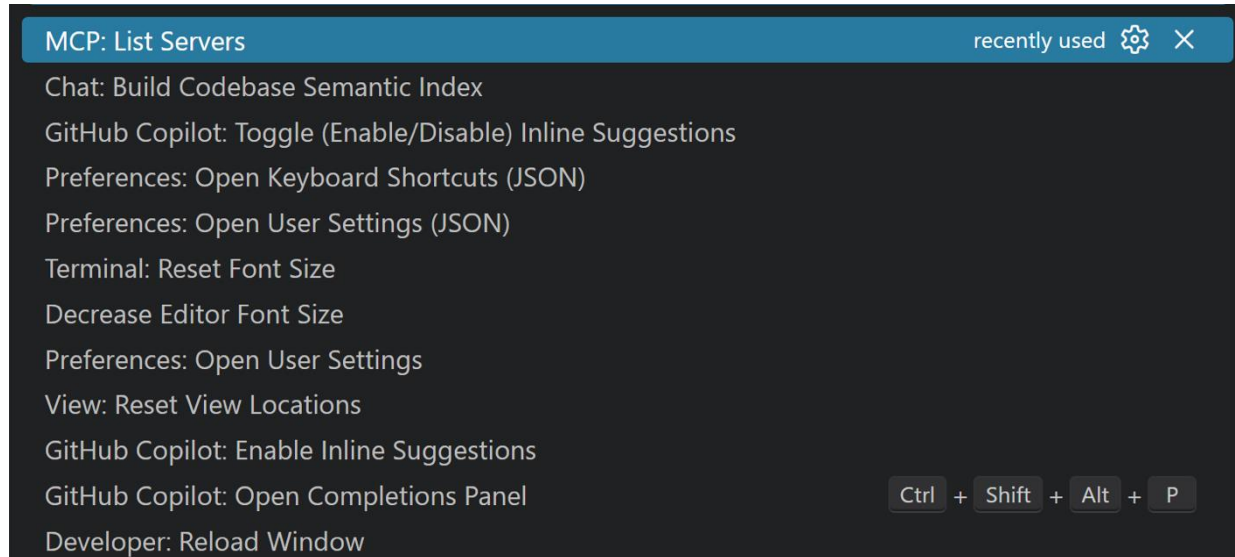


In the Extension sidebar, there is a dedicated section labeled "MCP SERVERS - INSTALLED" that lists all MCP servers currently configured on your machine.

To browse servers you have not yet installed, type `@mcp` in the Extensions search bar at the top — this filters the view to show only MCP-related entries.

You can narrow it further by adding a name after it, for example `@mcp markdown`, to find a specific server.

MCP SERVERS: LIST



You can list the running MCP servers.

In VS Code, open the Command Palette with Ctrl+Shift+P (Windows/Linux) or Cmd+Shift+P (Mac), then type:

MCP: List Servers

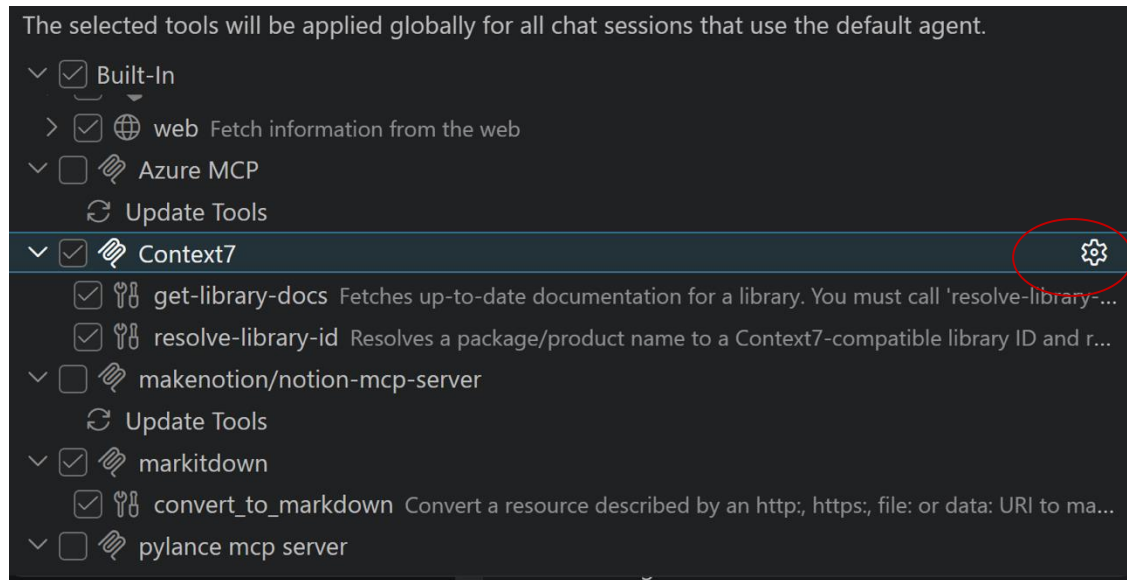
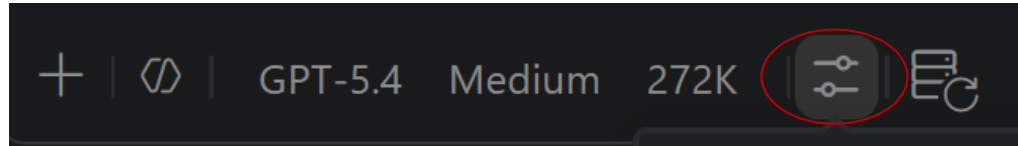
CONFIGURE TOOLS

The Configure Tools icon in the GitHub Copilot Chat input bar allows for managing which tools and MCP servers are available to Copilot in Agent mode. From here you can toggle tools on or off, and jump straight into configuration for any MCP Server.

Workflow:

- Click the Configure Tools icon in the Copilot Chat input bar to open the tools panel, which lists all available tools
- Find the MCP server you want in the list and click the gear icon to the right of its name
- The `mcp.json` file opens directly, where you can edit the server's configuration such as its command, arguments, and environment variables

CONFIGURE TOOLS - 2



Here is how to open the configuration (mcp.json) of any MCP Server from the Configure tools pane.

POP QUIZ: MCP SERVER FILES

What are the essential files for any MCP server?



10 MINUTES

SAMPLE CODE

```
var (
    background = gxui.Gray10
    cmd *cobra.Command
    files []string
)

func init() {
    cmd = &cobra.Command{
        Use: "gedit [files...]",
        Run: func(cmd *cobra.Command,
            files []string) {
            gl.StartDriver(uiMain,
                files),
        },
    }
}
```

The sample code presented in this course is intended to illustrate concepts and techniques, not to serve as production-ready software. To keep examples clear and focused, some code samples may omit error handling, panic recovery, input validation, logging, security controls, and other robustness features.

In many cases, brevity and instructional clarity take precedence over completeness.

LABS

Learning is better when hands-on. Some of the modules have a companion lab reinforcing a kinesthetic learning experience.

- Labs review and reinforce important concepts
- Don't be surprised - many of the labs extend the topics introduced in the module
- The labs offer an opportunity for real-world experience
- Pair programming can be effective when working on the labs

COURSE AGENDA

1. MCP Protocol
2. GitHub Copilot
3. Streamable HTTP
4. FastMCP
5. Error Handling
6. Async / await
7. MCP Inspector
8. Logging
9. JSON Web Tokens (JWT)
10. OAuth2
11. MCP Clients
12. Node.js

YOUR CLASS!

Yes, this is your class. What does this mean? You determine the value.

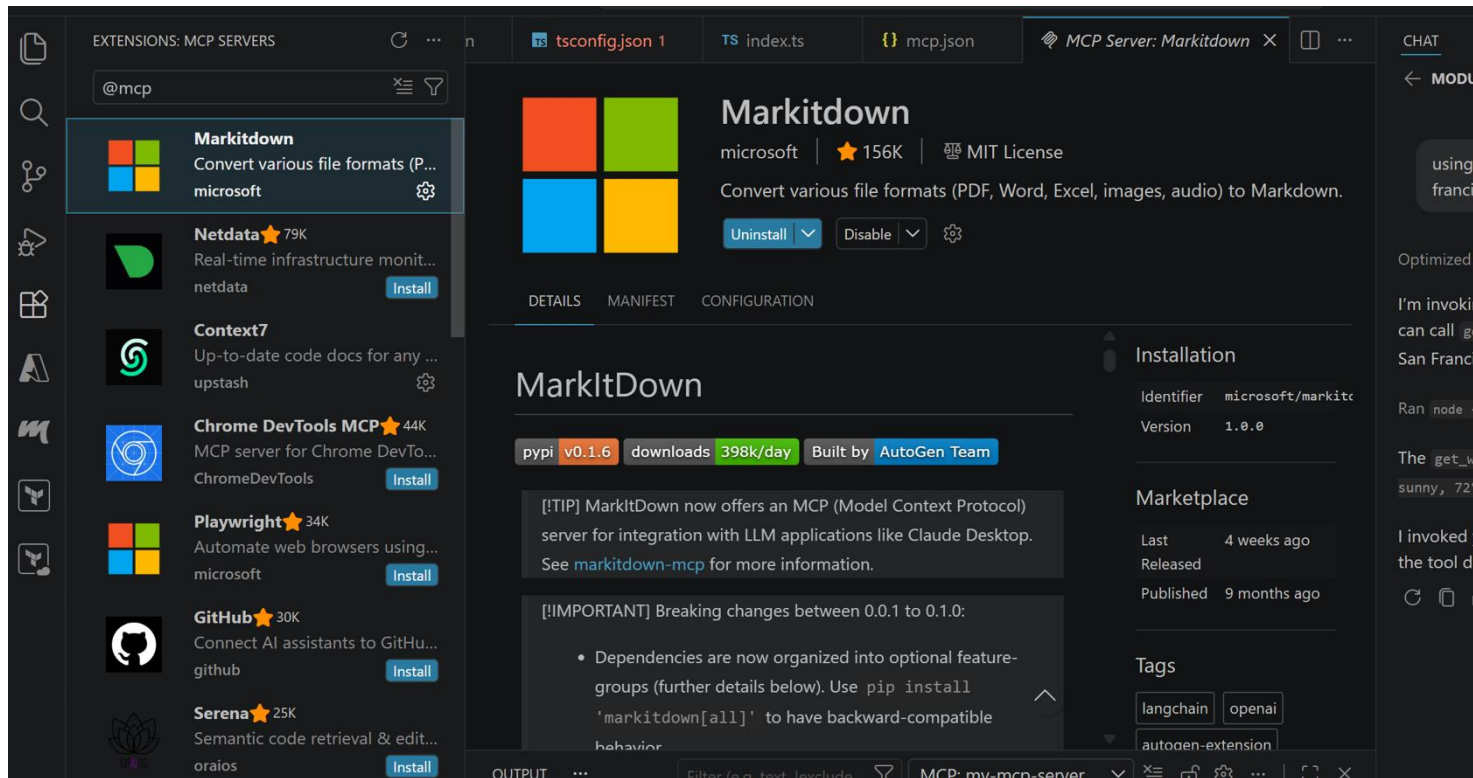
- What is the most important ingredient of class – your participation!
- Your feedback and questions are always welcomed.
- There is no protocol in class. Speak up anytime!
- We welcome your comments during and after class. Just email dmarshall@innovationinsoftware.com.

LAB 1 – MARKITDOWN



LAB

In this lab you will install and configure the Markdown MCP server in VS Code, verify it is running, and use GitHub Copilot in Agent mode to convert a text file to Markdown.



PREREQUISITE: INSTALL UVX

Before the Markdown MCP server can run, *uvx* must be installed on your machine. The Markdown MCP server is a Python-based tool, and VS Code launches it using *uvx* — a fast Python package runner that downloads and runs Python tools in isolated environments without requiring you to manually create a virtual environment or install the package globally.

When VS Code starts the Markdown server, it executes the command *uvx markdown-mcp* behind the scenes — so if *uvx* is not present on your system, the server will fail to start and show an error status in the MCP: List Servers view.

```
winget install --id=astral-sh.uv -e
```

STEP 1 - 2

Step 1: Open the Lab Workspace

Open VS Code and navigate to the lab01 folder inside your labs directory. Open it as your workspace (File → Open Folder → lab01). Once open, notice the hello.txt file at the root of the workspace — this is the file you will be converting to markdown (.md).

Step 2: Verify Markdown is Installed

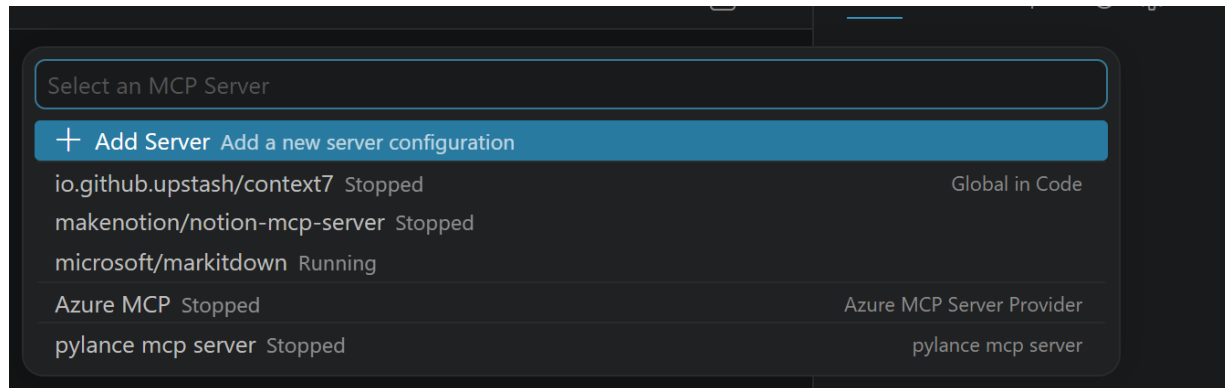
Open the Extensions sidebar by pressing Ctrl+Shift+X (Windows/Linux) or Cmd+Shift+X (Mac). Scroll to the bottom of the panel to find the MCP SERVERS - INSTALLED section. Confirm that Markdown appears in the list. If it is not there, search @mcp markdown in the Extensions search bar and install it.

STEP 3 – ENABLE AND START THE SERVER

In the MCP SERVERS - INSTALLED section, click the gear icon to the right of the Markdown server entry. From the gear menu, select:

- Enable for this Workspace — this registers the Markdown MCP server specifically for the current workspace, writing its configuration into the workspace's mcp.json file. This means the server will only be active when you have this workspace open, keeping your other projects unaffected.
- Start Server — this launches the Markdown MCP server process so it is running and ready to accept tool calls from Copilot.

STEP 4 – CONFIRM THE SERVER IS RUNNING

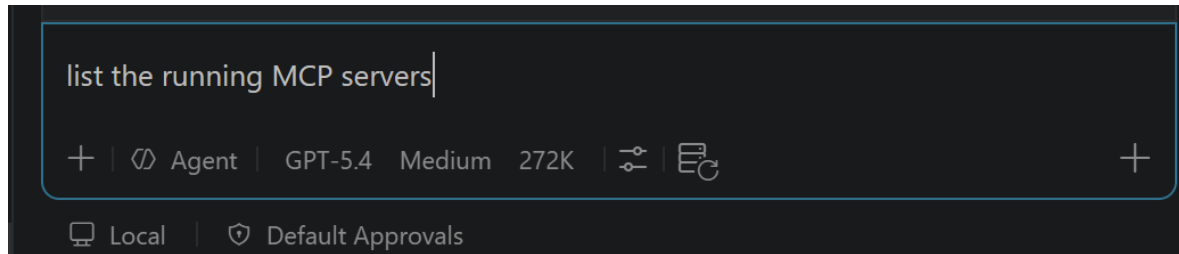


Open the Command Palette with Ctrl+Shift+P (Windows/Linux) or Cmd+Shift+P (Mac) and type:

MCP: List Servers

The Markdown server should appear in the list with a running status. If it shows as stopped, return to Step 3 and select Start Server from the gear menu of the MCP extension.

STEP 5 – CONFIRM THE SERVER IN COPILOT CHAT

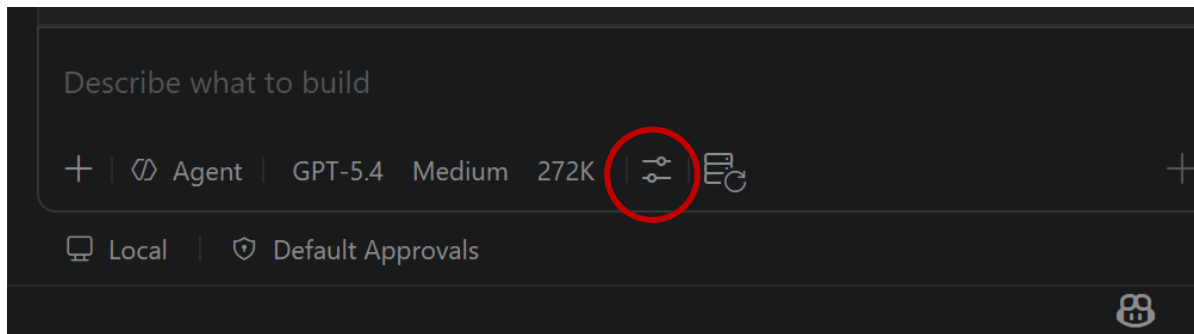


Open GitHub Copilot Chat and switch to Agent mode. In the chat input, enter the following prompt:

list the running MCP servers

Copilot will respond with the currently active MCP servers, and Markdown should be listed among them, confirming it is available.

STEP 6 – REVIEW THE MARKDOWN TOOLS



Click the Configure Tools icon (the wrench icon near the Copilot Chat input bar) to open the tools panel. Locate the Markdown server in the list — you will see it exposes a single tool:

`convert_to_markdown`

This tool converts a text file to a Markdown format.

STEP 7

In the GitHub Copilot Chat input (still in Agent mode), enter the following prompt:

using the markdown server convert file:\hello.txt to
hello.md markdown file.

When Copilot presents a confirmation prompt showing the tool call it is about to make, click Allow. The Markdown server will convert hello.txt and save the output as hello.md in your workspace root directory.

Open the hello.md file in VS Code and select Preview to confirm the content was correctly converted to Markdown format.

Lab completed

“Develop a passion for learning.”



MCP PROTOCOL

MCP INSPECTOR



INTRODUCTION



Welcome to the Model Context Protocol (MCP), an open standard that enables AI applications to connect to external tools, data sources, and services through a consistent interface. Instead of building custom integrations for every system, MCP provides a common protocol that allows AI clients and MCP servers to work together seamlessly.

In this module, you will learn the core concepts of MCP, including its architecture, JSON-RPC messaging, session lifecycle, and discovery mechanisms. You will also gain hands-on experience using the MCP Inspector.

MCP – OPEN STANDARD

Before MCP, every AI application needed a custom integration for every external tool or service — a sprawling, unmaintainable web of one-off connectors. MCP solves this by defining a single, open standard that any AI client and any external server can implement, so they can communicate without knowing anything specific about each other in advance.

- Open standard — not owned by any single vendor
- Based on JSON-RPC, a lightweight remote procedure call protocol
- Works across any language, platform, or environment
- One integration pattern instead of many custom ones

MCP ROLES

MCP defines three roles in every interaction:

- MCP Host — the AI application or agent that wants to use external capabilities (e.g. Claude, ChatGPT, VS Code Copilot)
- MCP Client — the component inside the host that speaks the MCP protocol and manages the connection
- MCP Server — the external system that exposes tools, resources, or prompts over MCP (e.g. Markdown, GitHub, Notion)

The client and server communicate over a transport layer — either stdio (local process) or HTTP (remote).

WIRE PROTOCOL UNDER MCP

MCP uses JSON-RPC 2.0 as its messaging format. Every message between client and server is a JSON object with a predictable structure:

- method — the name of the operation being requested (e.g. tools/list)
- params — the input data for that operation
- id — a unique identifier so responses can be matched to requests
- result / error — the server's response to the request

JSON-RPC is stateless, transport-agnostic, and human-readable — making it easy to debug, log, and implement in any programming language.

EXAMPLE REQUEST

```
{  
  "jsonrpc": "2.0",  
  "id": "req-001",  
  "method": "tools/list",  
  "params": {  
    "cursor": null  
  }  
}
```

This is the tools/list request to ask a server what tools it exposes.

"jsonrpc": "2.0" declares the protocol version being used, the id field links the response back to this specific request, and cursor (the bookmark): null in the cursor means start from the beginning.

RESPONSE

```
{
  "jsonrpc": "2.0",
  "id": "req-001",
  "result": {
    "tools": [
      {
        "name": "web_search",
        "description": "Search the web for current information",
        "inputSchema": {
          "type": "object",
          "properties": {
            "query": { "type": "string" }
          },
          "required": ["query"]
        }
      }
    ]
  }
}
```

RESPONSE - 2

This is a response to a tools/list request.

- The id is "req-001", same as your original request's id. That's the matching ticket number — proves this response goes with that exact request.
- It has a "result" field (not an "error" field), meaning the request succeeded.
- Inside result, there's a "tools" array — that's the list of tools the server is offering. That's the standard shape for a tools/list response.
- One thing missing: there's no "nextCursor" field. That just means this is the last (or only) page — no bookmark needed because there's nothing more to fetch. In this case there's only one tool (web_search), so a single page makes sense.

RESPONSE - 3

When a server responds, it wraps its payload inside a result field — or an error field if something went wrong. In the case of tools/list, the result contains an array of tool definitions, each with three key properties:

- name — the identifier used to call the tool later
- description — a human-readable explanation the LLM reads to decide whether to use the tool
- inputSchema — a formal JSON Schema contract specifying exactly what arguments the tool expects, their types, and which are required

CONTRACT

This contract is what allows an LLM to confidently construct a valid tool call without guessing.

This two-step pattern — discover tools first, then call them — is central to how MCP works:

- The client sends tools/list to learn what tools are available
- The LLM reads the descriptions to decide which tool fits the task
- A tools/call request is sent with arguments that conform to the tool's inputSchema

MCP SERVER SESSION LIFECYCLE



A predictable lifecycle means any MCP client can work with any MCP server without prior knowledge of its implementation.

Every interaction between an MCP client and server follows the same lifecycle:

Initialize — handshake, version negotiation, capability exchange

Discovery — client asks what tools, resources, and prompts are available

Invocation — client calls tools, reads resources, or retrieves prompts

Shutdown — session is cleanly terminated

INITIALIZE REQUEST

```
{  
  "jsonrpc": "2.0",  
  "id": 1,  
  "method": "initialize",  
  "params": {  
    "protocolVersion": "2025-06-18",  
    "capabilities": {  
      "roots": { "listChanged": true },  
      "sampling": {}  
    },  
    "clientInfo": {  
      "name": "ExampleClient",  
      "version": "1.0.0"  
    }  
  }  
}
```

There is a handshake that starts every session. The very first message in any MCP session is the initialize request, sent by the client. It contains:

- The MCP protocol version the client supports
- The client's capabilities — what features it can handle
- Client metadata — name and version of the client application

INITIAL RESPONSE

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "protocolVersion": "2025-06-18",
    "capabilities": {
      "tools": { "listChanged": true },
      "resources": { "subscribe": true, "listChanged": true },
      "prompts": { "listChanged": true },
      "logging": {}
    },
  },
  "serverInfo": {
    "name": "example-server",
    "version": "1.0.0"
  }
}
```

The server responds with its own protocol version, capabilities, and its metadata. This handshake ensures both sides agree on what is supported before any real work begins — preventing errors caused by feature mismatches.

CAPABILITIES

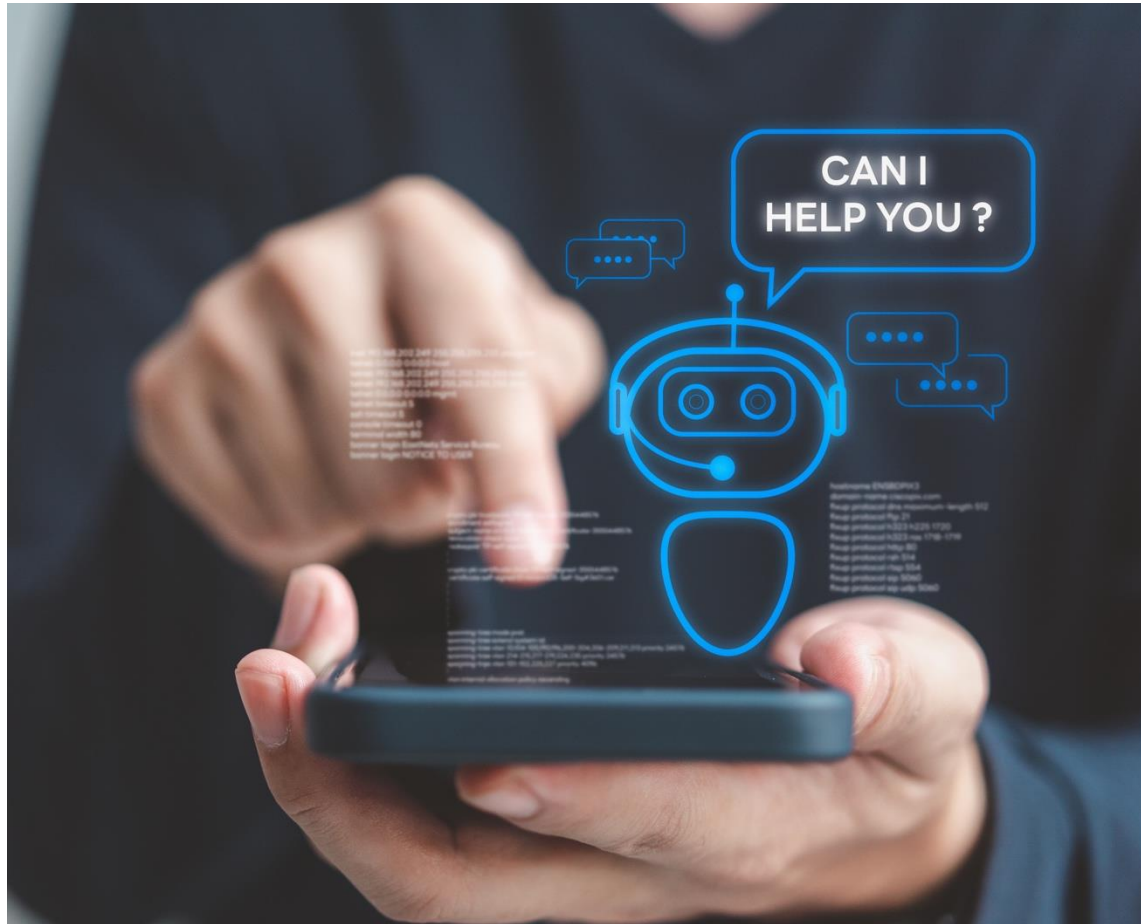


During initialization, both sides advertise their capabilities — the optional features they support beyond the core protocol. Most of this is self-explanatory except:

Client capabilities:

- sampling — can AI be asked to assist
- roots — what folders are accessible

SAMPLING



Sampling means the server can ask the client "hey, can you think about this for me?" The server is doing its job — maybe running a tool — but hits a moment where it needs an AI brain to make a decision or generate some text. Instead of having its own AI, it just borrows the client's.

Example: A server is processing a long document and needs to summarize a section. It asks the client to run that summarization using its LLM, then continues with the result.

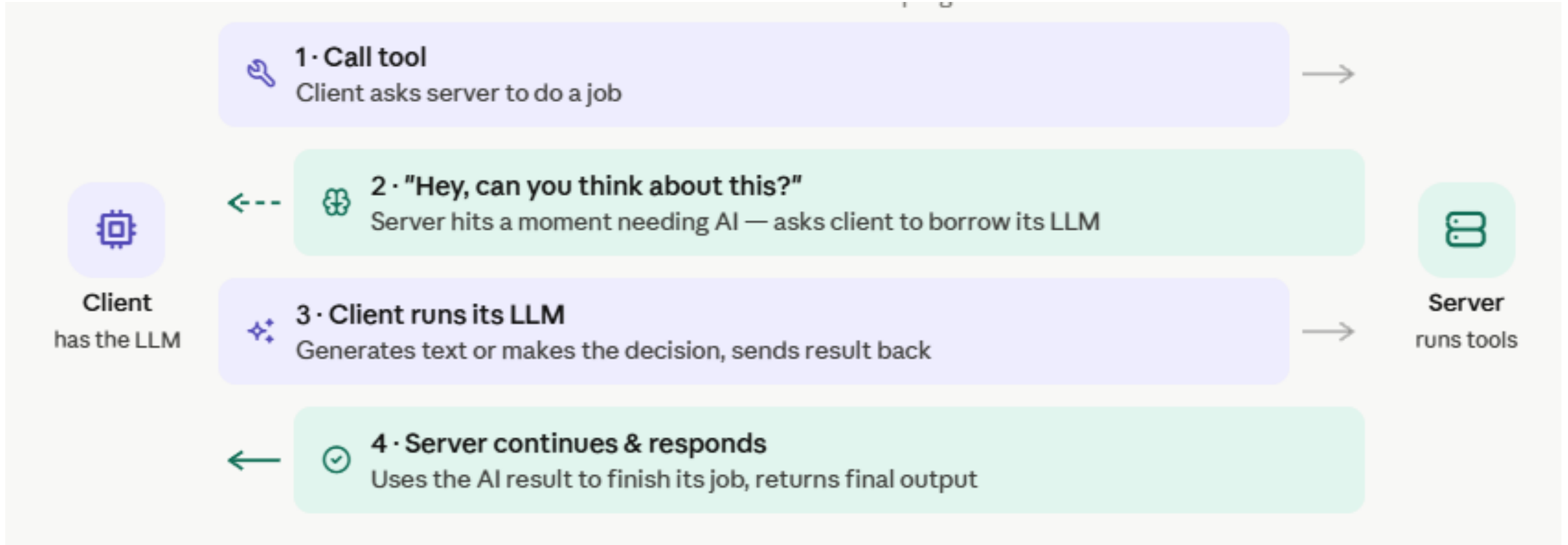
ROOTS



Roots is how the client tells the server "here are the folders on my machine you're allowed to see."

When the client advertises the roots capability, it shares a list of file system paths — like `/Users/me/project` — that the server can reference. This lets the server do things like read files, understand the project structure, or search for context, without having unrestricted access to the entire file system.

CAPABILITIES



TOOLS

After initialization, the client sends a `tools/list` request to discover all tools the server exposes. The server responds with an array of tool definitions, each containing:

- `name` — the unique identifier for the tool (e.g. `convert_to_markdown`)
- `description` — a human-readable explanation of what the tool does
- `inputSchema` — a JSON Schema object defining the expected parameters

The AI model uses this schema to understand how to call the tool correctly — including what arguments are required, their types, and any constraints. This is how GitHub Copilot knows to call `convert_to_markdown` with a `uri` parameter.

DATA RESOURCES

A resources/list request asks the server what data it can expose for reading. Resources represent external content the AI can access — files, database records, API responses, configuration values, and more. Each resource entry includes:

- uri — a unique identifier (e.g. file:///data/report.pdf)
- name — a human-readable label
- mimeType — the format of the content (e.g. text/plain, application/json)
- description — what the resource contains

Resources are read-only by design — they provide context to the AI without allowing mutation through the resource API.

PROMPTS

A `prompts/list` request retrieves all prompt templates the server exposes. Prompts in MCP are not free-form text — they are structured templates with defined input variables that produce consistent, reusable messages for the AI. Each prompt entry includes:

- `name` — the unique identifier for the prompt
- `description` — what the prompt is used for
- `arguments` — the variables the caller must supply

Example: a server might expose a `summarize_document` prompt that accepts a `document_uri` argument and returns a structured summarization request ready for the AI model to execute.

PROMPTS

```
# Prompt: review-code
@mcp.prompt(
    "review_code",
    title="Code Review",
    description="Review code for best practices and potential issues",
)

def review_code(language: str, code: str) -> str:
    """
    Args:
        language: Programming language of the code
        code: The code to review
    """
    return f"Review this {language} code for best practices:\n\n{code}"
```

CALLING TOOLS

```
{  
  "jsonrpc": "2.0",  
  "id": "req-003",  
  "method": "tools/call",  
  "params": {  
    "name": "web_search",  
    "arguments": { "query": "latest MCP specification" }  
  }  
}
```

Executing a tool on the server

Once the client knows what tools are available, it can invoke one using tools/call. The request includes:

- name — the tool to invoke (must match a name from tools/list)
- arguments — a JSON object matching the tool's inputSchema

CALLING TOOLS - 2

```
{
  "jsonrpc": "2.0",
  "id": "req-003",
  "result": {
    "content": [
      {
        "type": "text",
        "text": "The latest MCP specification was released"
      }
    ],
    "isError": false
  }
}
```

The server executes the tool and returns a content array — one or more content blocks of type text, image, or resource.

The AI model receives this output and incorporates it into its response. For example, calling `convert_to_markdown` with `{"uri": "file:///hello.txt"}` returns the converted Markdown text as a text content block.

WALKTHROUGH – END TO END



This is the walkthrough of an MCP client and MCP server conversation, as defined by the MCP Protocol.

INITIALIZE MCP SERVER

Client send this JSON to initialize a conversation.

```
{
  "jsonrpc": "2.0",
  "id": "req-001",
  "method": "initialize",
  "params": {
    "protocolVersion": "2024-11-05",
    "clientInfo": { "name": "my-client", "version": "1.0" },
    "capabilities": { "sampling": {}, "roots": {} }
  }
}
```

Server responds:

```
{
  "jsonrpc": "2.0",
  "id": "req-001",
  "result": {
    "protocolVersion": "2024-11-05",
    "serverInfo": { "name": "my-server", "version": "1.0" },
    "capabilities": { "tools": {} }
  }
}
```


LIST THE TOOLS

Client requests the available tools from the MCP server.

```
{  
  "jsonrpc": "2.0",  
  "id": "req-002",  
  "method": "tools/list",  
  "params": {}  
}
```

The server responds:

```
{  
  "jsonrpc": "2.0",  
  "id": "req-002",  
  "result": {  
    "tools": [  
      {  
        "name": "web_search",  
        "description": "Search the web for current information",  
        "inputSchema": {  
          "type": "object",  
          "properties": {  

```

CALL THE WEB_SEARCH TOOL

The client calls a tool.

```
{
  "jsonrpc": "2.0",
  "id": "req-003",
  "method": "tools/call",
  "params": {
    "name": "web_search",
    "arguments": { "query": "latest MCP specification" }
  }
}
```

The server responds:

```
{
  "jsonrpc": "2.0",
  "id": "req-003",
  "result": {
    "content": [
      {
        "type": "text",
        "text": "The latest MCP specification was released in November 2024, introdu
      }
    ],
    "isError": false
  }
}
```

ERROR RESPONSE

How does the server respond with an error?

```
{  
  "jsonrpc": "2.0",  
  "id": "req-003",  
  "result": {  
    "content": [  
      {  
        "type": "text",  
        "text": "Search failed: unable to reach the network"  
      }  
    ],  
    "isError": true  
  }  
}
```

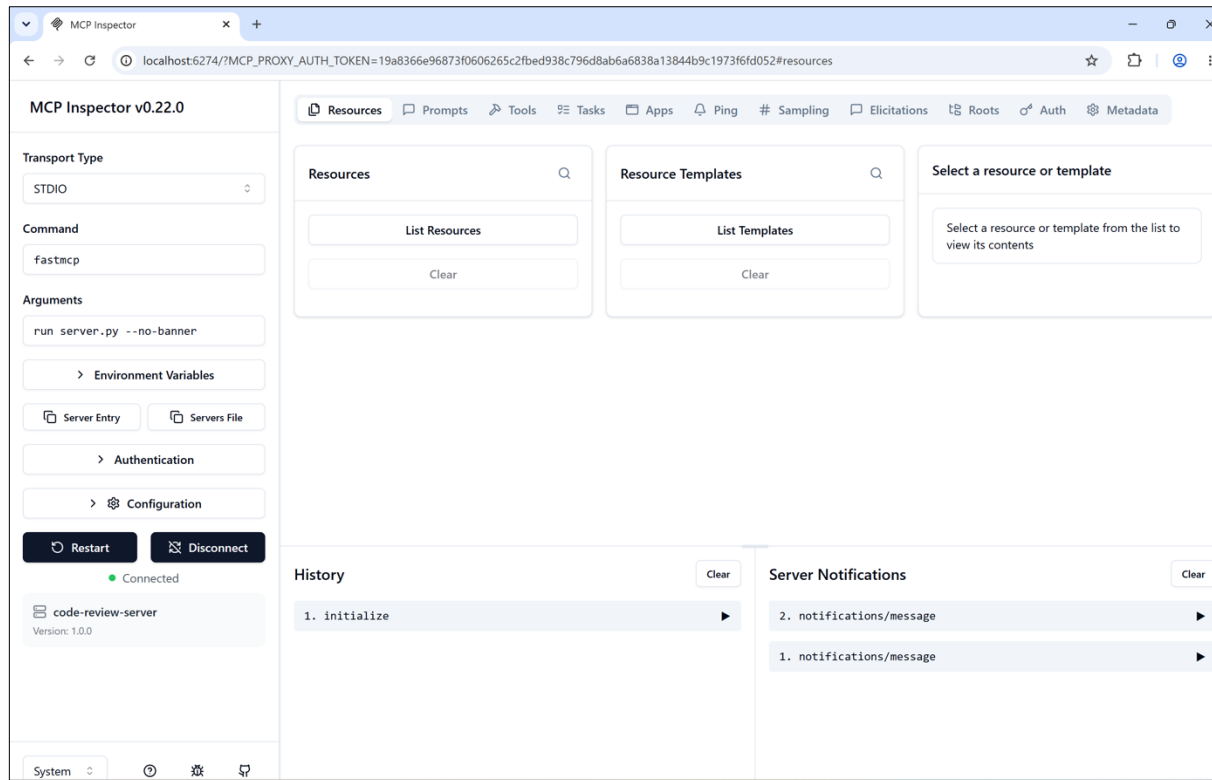


MCP INSPECTOR

MCP Inspector is an interactive UI and proxy tool that allows developers to connect to and debug MCP servers. It makes debugging much easier — you can see what tools your server offers, what inputs and outputs look like, and where things break.

It consists of two main components that work together: the MCP Inspector Client (MCPI), a React-based web UI that provides an interactive interface for testing and debugging, and the MCP Proxy (MCPProxy), a Node.js server that acts as a protocol bridge connecting the web UI to MCP servers via various transport methods — stdio, SSE, or streamable HTTP.

MCP INSPECTOR



It's great for watching the MCP protocol at work — you can see every request and response in real time as JSON, observe the full lifecycle of a tool call (from invocation to result),

CONNECTION PANEL

The screenshot shows the 'MCP Inspector v0.22.0' interface. It features a 'Transport Type' dropdown set to 'STDIO'. Below it is a 'Command' input field containing 'fastmcp'. The 'Arguments' section contains 'run server.py --no-banner'. There are expandable sections for 'Environment Variables', 'Authentication', and 'Configuration'. At the bottom of the settings are 'Restart' and 'Disconnect' buttons. A green dot indicates the connection status is 'Connected'. Below this, a server entry for 'code-review-server' (Version: 1.0.0) is listed. The footer includes a 'System' dropdown and three icons: a question mark, a refresh symbol, and a disconnect symbol.

Defines how to connect to an MCP server — it is where you specify everything needed to launch and connect to a server before you can inspect it. Think of it as the "setup" panel you fill out once before doing any testing.

It has three logical groupings:

- Connection Settings
- Advanced Options
- Connection Controls

TRANSPORT

Transport Type

STDIO

Command

fastmcp

Arguments

run server.py --no-banner

STDIO

Launches the server as a local subprocess. The Inspector communicates by writing JSON-RPC messages to standard input and reading responses from standard output. Best for locally running servers like FastMCP.

SSE (Server-Sent Events)

For remotely hosted servers over HTTP. The client sends requests via HTTP POST and the server streams responses back over a persistent Server-Sent Events connection.

STREAMABLE HTTP

Streamable HTTP is how an AI app (the client) and a tool server talk over the internet. The client sends every request to one single web address using HTTP POST — no need for different addresses for different things.

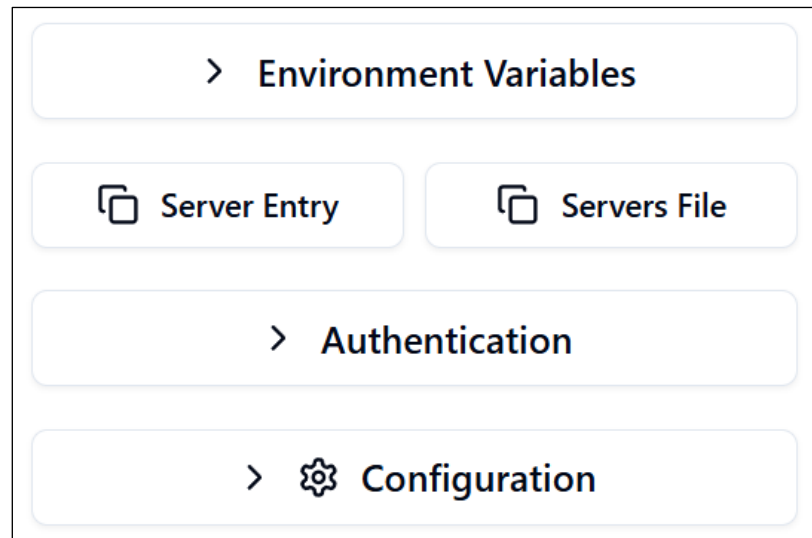
Once the server gets a request, it can answer two ways. Quick stuff gets a normal, one-shot response right away. But for tasks that take a while, the server can keep the connection open and send the answer in pieces over time — updating the client as it goes instead of leaving it waiting in silence. That's "streamable": the connection *can* stream updates when needed, but doesn't have to.

STREAMABLE HTTP - 2

Why it matters:

This replaced an older setup called SSE, which needed two separate addresses — one to send, one to receive — and made developers juggle both. Streamable HTTP simplifies that into one address that handles quick and slow answers alike, which is why it's now the standard for any new MCP server.

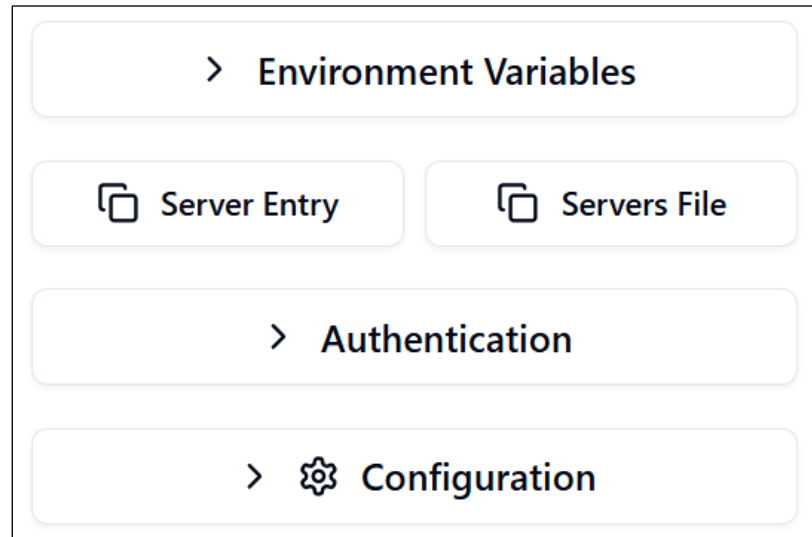
MISCELLANEOUS



Environment variables

A collapsible section where you define key-value pairs injected into the server process at launch time — such as API keys, database connection strings, or feature flags — without hardcoding them into the server code. For example, a GitHub MCP server would need a `GITHUB_TOKEN` variable here to authenticate with the GitHub API.

MISCELLANEOUS - 2



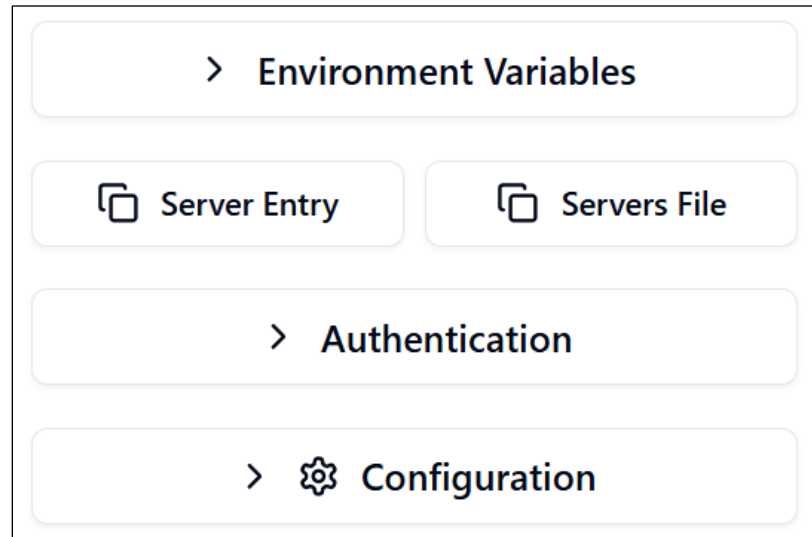
Server Entry

A button that lets you load a single server's configuration directly into the connection panel — pre-filling the Transport Type, Command, and Arguments fields from a saved entry.

Servers File

Similar to Server Entry but loads from a full servers configuration file — equivalent to an mcp.json file that defines multiple servers. This is useful when working with multiple MCP servers across a project.

MISCELLANEOUS - 3



Authentication

A collapsible section for configuring credentials needed to connect to secured MCP servers. Primarily used with SSE and Streamable SSE transports where the remote server requires an auth token or API key.

Configuration

A collapsible section for Inspector-level settings that control connection behavior — such as timeouts, retry logic, or protocol options. These affect how the Inspector manages the session rather than how the server runs.

SERVER ENTRY FILE

Here is an example of a Server Entry configuration in JSON format, as it would appear in an mcp.json file:

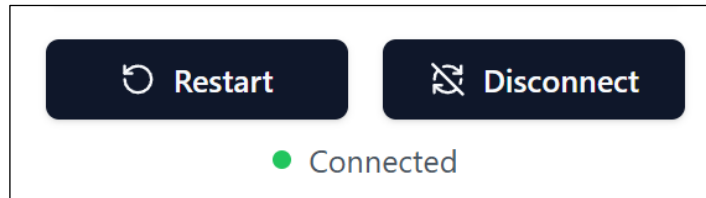
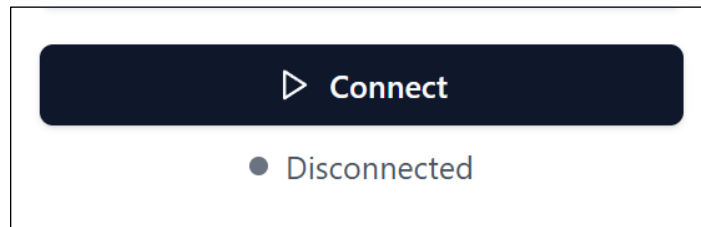
```
{
  "mcpServers": {
    "demo-server": {
      "command": "fastmcp",
      "args": ["run", "server.py", "--no-banner"],
      "env": {
        "GITHUB_TOKEN": "your-token-here",
        "DEBUG": "false"
      }
    }
  }
}
```

SERVERS FILE

A Servers File is essentially the same format as a Server Entry but contains multiple server definitions in a single file:

```
{
  "mcpServers": {
    "demo-server": {
      "command": "fastmcp",
      "args": ["run", "server.py", "--no-banner"],
      "env": {
        "DEBUG": "false"
      }
    },
    "markdown": {
      "command": "uvx",
      "args": ["markdown-mcp"],
      "env": {}
    },
    ...
  }
}
```

CONNECT



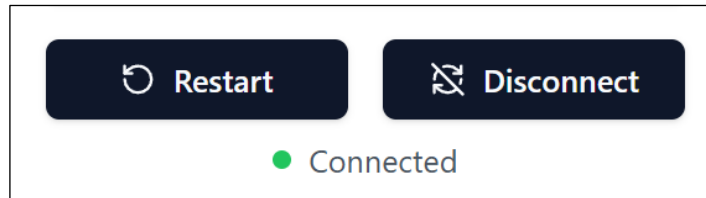
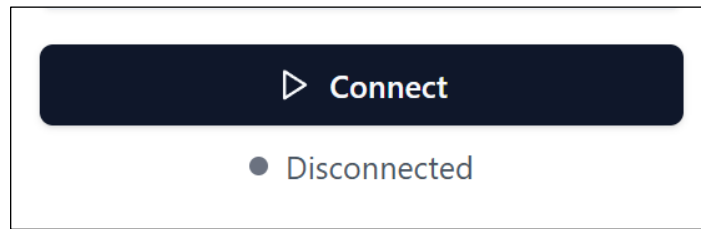
Connect

Initiates the MCP session by launching the server using the configured Command and Arguments. Behind the scenes it sends the initialize handshake, negotiating the protocol version and capabilities. Once successful, the green Connected indicator appears along with the server's name and version.

Disconnect

Terminates the active MCP session, shutting down the server subprocess (STDIO) or closing the HTTP connection. It will also reset the Inspector to its unconnected state.

CONNECT - 2



Restart

Stops and immediately relaunches the server without changing any connection settings — a Disconnect and Connect in one click. Most commonly used during development to reload the server after code changes in `server.py`, re-running the full initialize handshake to pick up the latest capabilities.

WALKTHROUGH – LIFECYCLE



In this walkthrough you will explore the MCP Inspector with the Math MCP Server.

STEPS 1 AND 2

Step 1: Open the Workspace

Open VS Code and navigate to Module 2. Open the math directory as your workspace (File → Open Folder → math). Confirm the folder is loaded as the active workspace in the Explorer panel.

Step 2: Start the Math MCP Server

Open the mcp.json file in the workspace. Locate the math-mcp server entry. Click the Start link above the MCP server name. Confirm the server is running in the configuration file and also opening the Command Palette (Ctrl+Shift+P) and running MCP: List Servers — math-mcp should show a running status.

STEPS 3 AND 4

Step 3: Test with GitHub Copilot Chat

Open GitHub Copilot Chat and switch to Agent mode. Enter the following prompt:

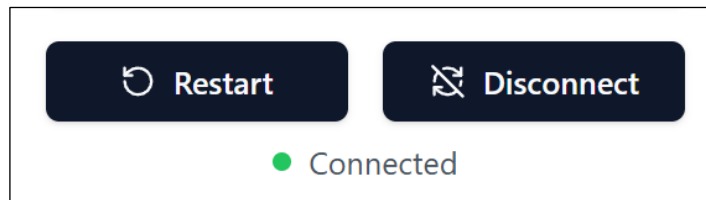
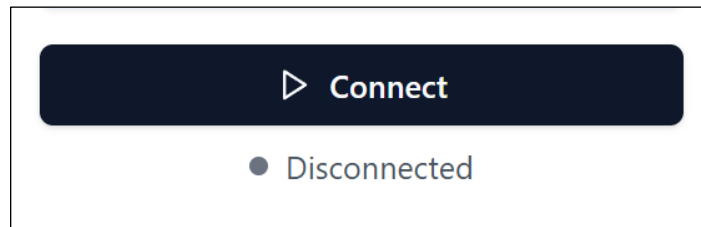
using the math-mcp server return what is the answer.

When Copilot presents the tool confirmation dialog, click Allow. Review the result returned in the chat window.

Step 4: Stop the Math MCP Server

Return to the mcp.json file. Click the Stop hyperlink. Confirm it is no longer running via MCP: List Servers in the Command Palette.

MCP INSPECTOR



Open a terminal or command prompt outside of VS Code. Navigate to the directory containing `server.py` for the `math-mcp` server:

Launch the MCP Inspector by running:

```
fastmcp dev server.py
```

The MCP Inspector will open in your browser.

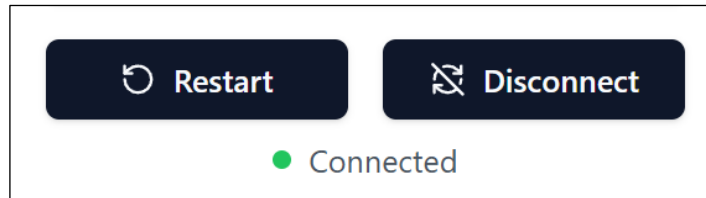
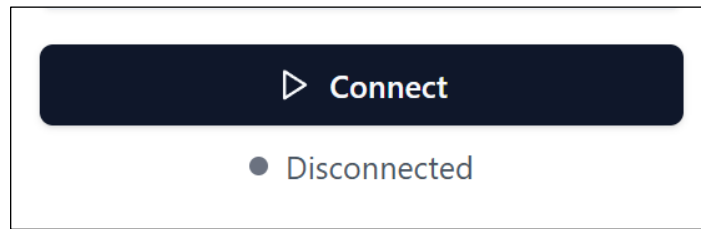
REVIEW - STEP 6

Before connecting, take a moment to review the Connection Panel on the left side of the MCP Inspector. Examine each setting:

- Transport Type — is it set to STDIO?
- Command — what executable is listed?
- Arguments — what arguments are passed to the command?
- Environment Variables, Authentication, Configuration — are any values indicated for this server?

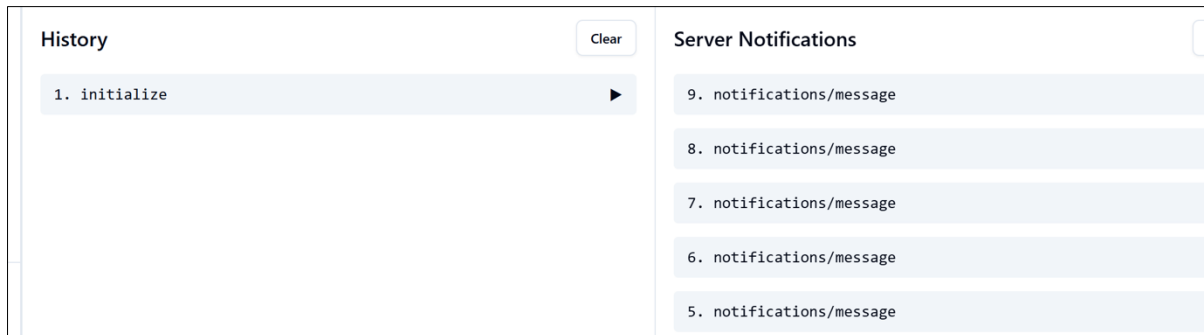
Make sure you understand what each setting does before proceeding.

CONNECT – STEP 7



Click the Connect button at the bottom of the Connection Panel. The MCP Inspector toolbar should appear across the top of the right panel, with panes for Tools, Resources, Prompts, and History. The Connected indicator (green) should appear along with the server name below the buttons.

CONNECT – STEP 8



Click the History tab in the toolbar. Locate the initialize message in the history list and click on it. Review both sides of the handshake:

- Client Request — notice the protocol version and capabilities the Inspector advertised to the server
- Server Response — notice the protocol version and capabilities the math-mcp server returned

This is the Initialize handshake for the MCP server as described earlier in the course.

STEPS 9 AND 10

Step 9: List Available Tools

From the toolbar, select the Tools tab. Click the List Tools button. The Inspector sends a tools/list request to the server and displays the tools the math-mcp server exposes. You should see the add tool history appear in the list.

Step 10: Run the Add Tool

Click on the add tool. An input panel appears on the right side with fields for a and b. Enter a number in each field. Scroll down and click the Run Tool button. The Inspector sends a tools/call request to the math-mcp server and displays the result.

TOOLS/CALL – STEP 11

History	Clear	Server Notifications	Clear
3. tools/call	▶	1. notifications/message	▶
2. tools/list	▶		
1. initialize	▶		

Click the History tab and locate the tools/call message. Click on it and review:

- Client Request — notice the tool name (add) and the arguments (a and b) that were sent
- Server Response — notice the content block returned by the server

Do you see the answer to your addition in the server response?

STEPS 12 AND 13

Step 12: Find the Data Resource

Click the Resources tab in the toolbar and click List Resources. Look through the available resources for `data://answer`. Click on it and select Read Resource to retrieve its content.

Step 13: Disconnect

Return to the Connection Panel on the left and click the Disconnect button. The green Connected indicator will disappear and the Inspector will reset to its unconnected state. The math-mcp server process is now shut down.

RETROSPECTIVE



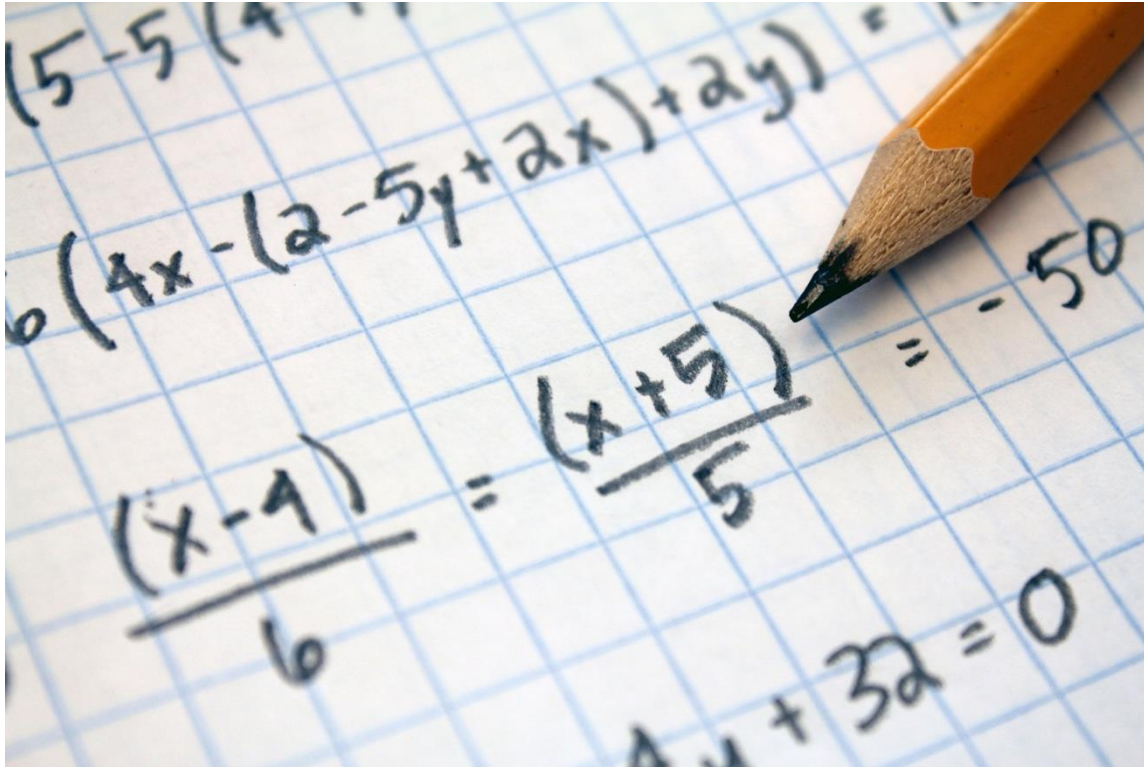
In this module, you learned how MCP standardizes communication between AI applications and external systems. You explored MCP roles, the initialization handshake, capability negotiation, tool discovery, resources, prompts, and tool execution.

You also used the MCP Inspector to observe protocol messages and interact with an MCP server. These concepts provide the foundation needed to build, test, and troubleshoot MCP-based applications and integrations.

LAB 2 – MCP CLIENT

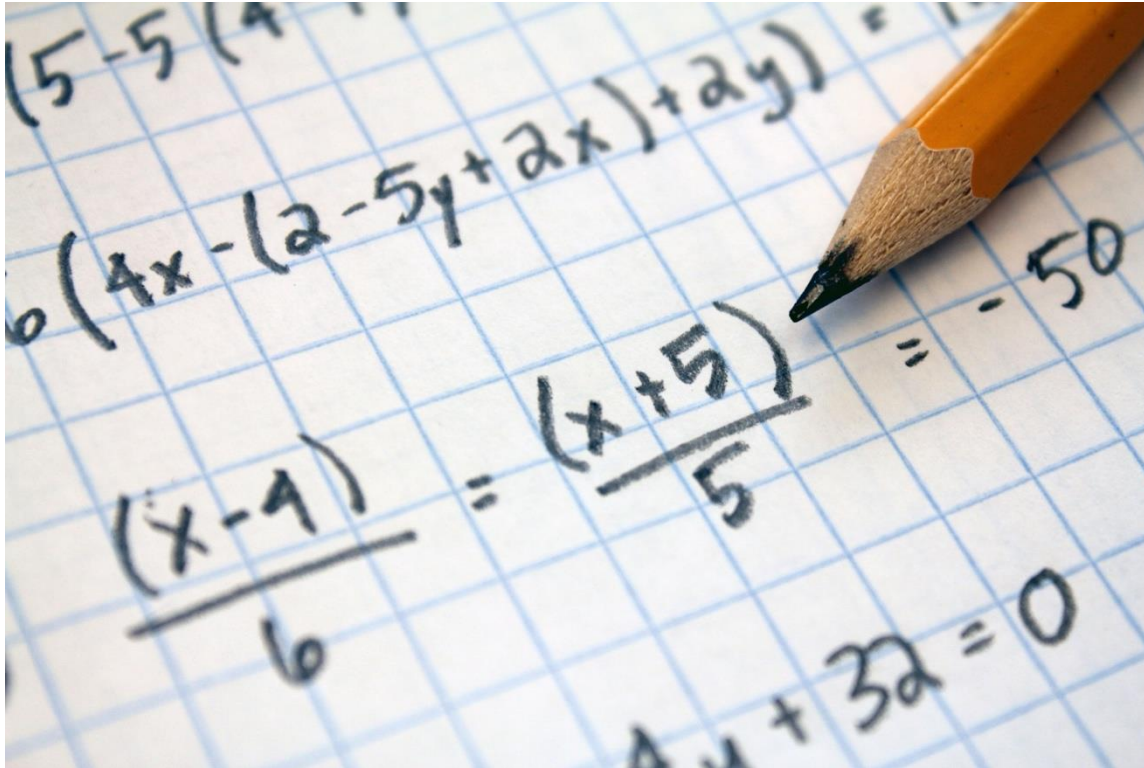


GOAL



In this lab you will explore a working MCP client and server, review the source code, extend the math MCP server with new tools, and call them from the client.

GETTING STARTED – STEP 1



Open VS Code and navigate to the labs directory, then into lab02. Inside lab02, open the add_client folder as your workspace (File → Open Folder and then select the add_client folder).

DIRECTORY LAYOUT – STEP 2

Before proceeding, familiarize yourself with the lab directory layout:

```
labs/  
├── lab02/  
│   ├── add_client/  
│   │   ├── math_server/  
│   │   │   ├── server.py    ← MCP server definition  
│   │   │   ├── client.py    ← MCP client that calls the server  
│   │   │   ├── mcp.json     ← MCP server configuration  
│   │   └── requirements.txt ← Python dependencies
```

- math_server/server.py — defines the math MCP server and its tools
- client.py — the MCP client that connects to the server and calls its tools
- mcp.json — tells VS Code and the Inspector how to launch the server
- requirements.txt — lists the Python packages needed to run the lab

STEP 3 AND 4

Step 3: Start the Math MCP Server

Locate the `math_server` directory — this is the root directory of the math MCP server. Open `mcp.json` in the workspace, find the `math-mcp` server entry, and start the server using one of these methods:

- Click the Start link for the `math-mcp` entry in `mcp.json`
- Open the Command Palette (Ctrl+Shift+P) and run MCP: Start Server

Step 4: Confirm the Server is Running

Open the Command Palette (Ctrl+Shift+P) and run:

MCP: List Servers

Confirm that `math-mcp` appears in the list with a running status. If it shows as stopped, return to Step 3 and try starting it again.

REVIEW SERVER.PY - STEP 5

Open `math_server/server.py` and read through the source code and comments carefully. As you review, look for and note the following:

- How the MCP server is initialized and named
- How the add tool is defined — its name, parameters, and return value
- How the `data://answer` resource is defined and what it stores
- How the server is launched at the bottom of the file

Take a few minutes to make sure you understand the server before moving on.

STEP 6 AND 7

Step 6: Review client.py

Open client.py and read through the source code and comments carefully. As you review, note the following:

- How the client connects to the math MCP server
- How it calls the add tool using tools/call with arguments a and b
- How the result is extracted from the server response and displayed

Understanding this client code is the foundation for the changes you will make in the next steps.

Step 7: Run the Client

Open a terminal in VS Code and run the client:

```
python client.py
```

ADD MATH METHODS - STEP 8

Open `math_server/server.py` and add three new tool definitions following the same pattern as the existing `add` tool. Using the `add` tool as your guide, write the following three tools:

- `subtract` — accepts two numbers and returns the result of subtracting `b` from `a`
- `multiply` — accepts two numbers and returns their product
- `divide` — accepts two numbers and returns the result of dividing `a` by `b`. Include a check to prevent division by zero.

Save the file when done.

SMOKE TEST - STEP 9

Restart the math-mcp server as a smoke test to confirm your new code has no syntax errors and the server starts without errors.

Run MCP: List Servers again to confirm the server is still showing a running status. If it shows an error, review your code in server.py for any syntax issues and restart again.

UPDATE CLIENT.PY - STEP 10

Open client.py and add calls to each of the three new tools following the same pattern as the existing add tool call. Write calls for the following operations:

- Multiply: 10×12
- Divide: $18 \div 3$
- Subtract: $25 - 5$

For each call, display the result to the console in the same format as the existing add output. Save the file when done.

TEST THE CLIENT - STEP 11

Run the client again in the terminal:

```
python client.py
```

Observe the output and confirm all four operations return the correct results:

- Multiply $10 \times 12 = 120$
- Divide $18 \div 3 = 6.0$
- Subtract $25 - 5 = 20$

If any result is incorrect, review the corresponding tool implementation in `server.py` and the tool call in `client.py`.

BONUS

If you have completed all steps early, extend `client.py` to read and display the `data://answer` resource.

Using the MCP resource API, retrieve the resource at `data://answer` and print its value to the console. To read the resource, add the following code to `client.py` after your last tool call:

```
# Read the data://answer resource  
resource = await session.read_resource("data://answer")
```

Run the client again to observe the value stored in the resource — notice that it reflects the result of the last tool call made to the server.

Lab completed

“Develop a passion for learning.”



ADVANCED

DETAILS AND WORKFLOW



INTRODUCTION



In this module, you will learn several advanced concepts that are essential for building production-ready MCP servers. Topics include asynchronous programming with `async` and `await`, Streamable HTTP transport, logging, error handling, and the Context object. These capabilities help MCP servers remain responsive, communicate effectively with clients, and handle failures gracefully.

You will explore both the underlying concepts and practical FastMCP examples. By the end of the module, you will understand how MCP servers manage concurrent work.

ASYNC



An async function in Python allows a program to start a task and then do other work while waiting for that task to finish.

This is useful for activities that spend a lot of time waiting, such as making web requests, reading from a database, or communicating with an MCP server.

Instead of sitting idle during the wait, Python can switch to another task and keep the application responsive.

COFFEE SHOP



A good analogy is a coffee shop worker. In a synchronous system, the worker starts a coffee machine and waits for it to finish before helping the next customer. In an asynchronous system, the worker starts the coffee, serves other customers, and returns when the machine signals that the coffee is ready.

For MCP servers, this allows a server to handle many requests efficiently without needing large numbers of threads.

NO ASYNC

```
def make_coffee():  
    print("Start coffee")  
    print("Wait while coffee brews")  
    print("Coffee done")  
  
def make_toast():  
    print("Start toast")  
    print("Wait while toast cooks")  
    print("Toast done")  
  
make_coffee()  
make_toast()
```

Coffee and toast:

You start the coffee.

You stand there doing nothing until the coffee is finished.

Only after the coffee is done do you start the toast.

That is simple, but inefficient. The wasted time is here:

Wait while coffee brews

During that waiting time, you could have started the toast.

ASYNC DETAILS

`async`

- Defines an asynchronous function (coroutine).
- The function can be paused and resumed.
- Calling it does not execute it immediately.
- Calling it creates a coroutine object.

`await`

- `await something()`
- Waits for another coroutine or async operation to complete.
- While waiting, control is returned to the event loop.
- Other async tasks may run during this time.
- When the awaited operation completes, execution resumes at the next line.

ASYNC DETAILS - 2

`asyncio.gather`

- Starts multiple coroutines.
- Allows them to make progress concurrently.
- Waits until all of them finish.
- Returns their results as a list.

```
await asyncio.gather(  
    make_coffee(),  
    make_toast()  
)
```


COFFEE - ASYNC

```
async def make_coffee():  
    print("Start coffee")  
    print("Coffee brewing (3  
seconds)...")  
    await asyncio.sleep(3)  
    print("Coffee done")
```

```
async def make_toast():  
    print("Start toast")  
    print("Toast toasting (2  
seconds)...")  
    await asyncio.sleep(2)  
    print("Toast done")
```

```
async def main():  
    start = time.perf_counter()  
  
    await asyncio.gather(  
        make_coffee(),  
        make_toast()  
    )  
  
    end = time.perf_counter()  
  
    print(f"\nTotal duration: {end  
- start:.2f} seconds")  
  
    asyncio.run(main())
```

ASYNCR - MCP

```
@mcp.tool()
async def greet(name: str, ctx: Context) -> str:
    """Greet a user by name"""

    await ctx.info(f"Starting greeting for: {name}")
    result = f"Hello, {name}!"
    await ctx.info("Done.")
    return result
```

Async is beneficial for MCP server tools because most tools spend much of their time waiting for external systems such as GitHub, databases, or LLM APIs.

Instead of leaving a thread idle while waiting for a response, async and await allow the server to pause that request and handle other incoming requests.

POP QUIZ: THREAD

What is a thread?



10 MINUTES

MCP AND THREADING

async/await runs everything on one single thread. It's not multiple threads taking turns — it's one thread that's smart about not sitting idle. When a task hits something that takes time (waiting for a network response, a file read, a database query), instead of the thread just freezing and doing nothing, it pauses that task, goes and works on something else that's ready to go, and comes back to the paused task once its wait is over.

This is exactly why async/await is great for I/O-bound work (API calls, database queries, network requests) — lots of waiting, little actual CPU work — but it does *nothing* to speed up CPU-heavy work like number crunching, because there's still only one thread doing the actual computing. For that, you'd need real multithreading or multiprocessing instead.

CHEF



Think of one chef in a kitchen cooking three dishes. The chef puts pasta water on to boil (`await asyncio.sleep()`), and *while it's boiling*, doesn't just stand there — they go chop vegetables for the salad.

When the water's ready, they come back to the pasta. One chef, one set of hands, never doing two things in the same instant — but no time wasted standing around either. That's `async/await`.

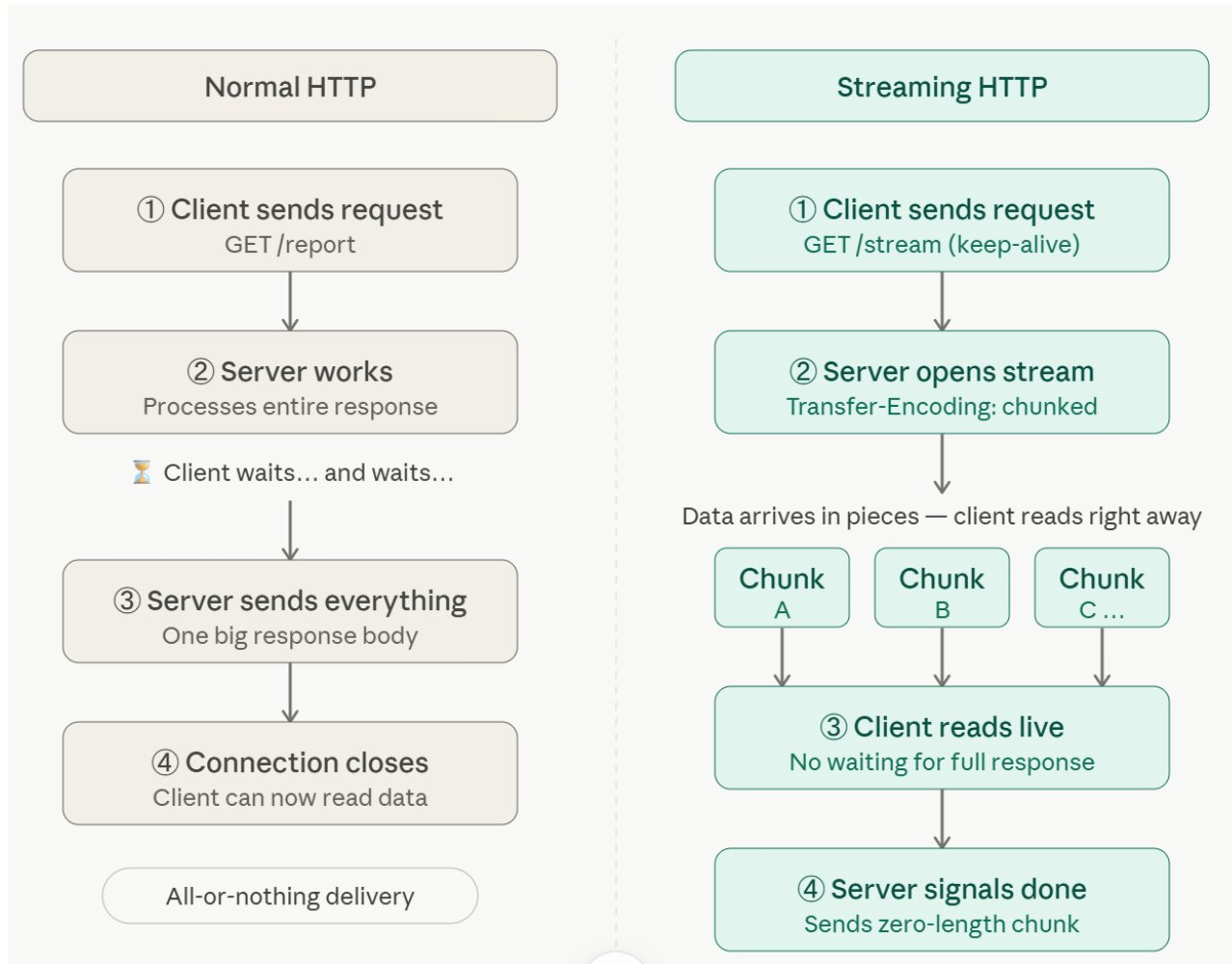
Multithreading would be hiring three chefs, each with their own station, genuinely cooking at the same time.

STREAMABLE HTTP

Streamable HTTP is the modern, network-capable option introduced in the March 2025 MCP spec update, replacing the older SSE transport with a cleaner, unified approach.

Instead of juggling two separate addresses like SSE required, Streamable HTTP exposes a single endpoint — typically `/mcp` — that handles everything. The client POSTs requests to it (like calling a tool), and when a response needs to stream back in pieces over time, that same connection stays open and delivers updates as they arrive. Quick answers come back immediately; slow ones stream in as they're ready. One address, two modes, no coordination required.

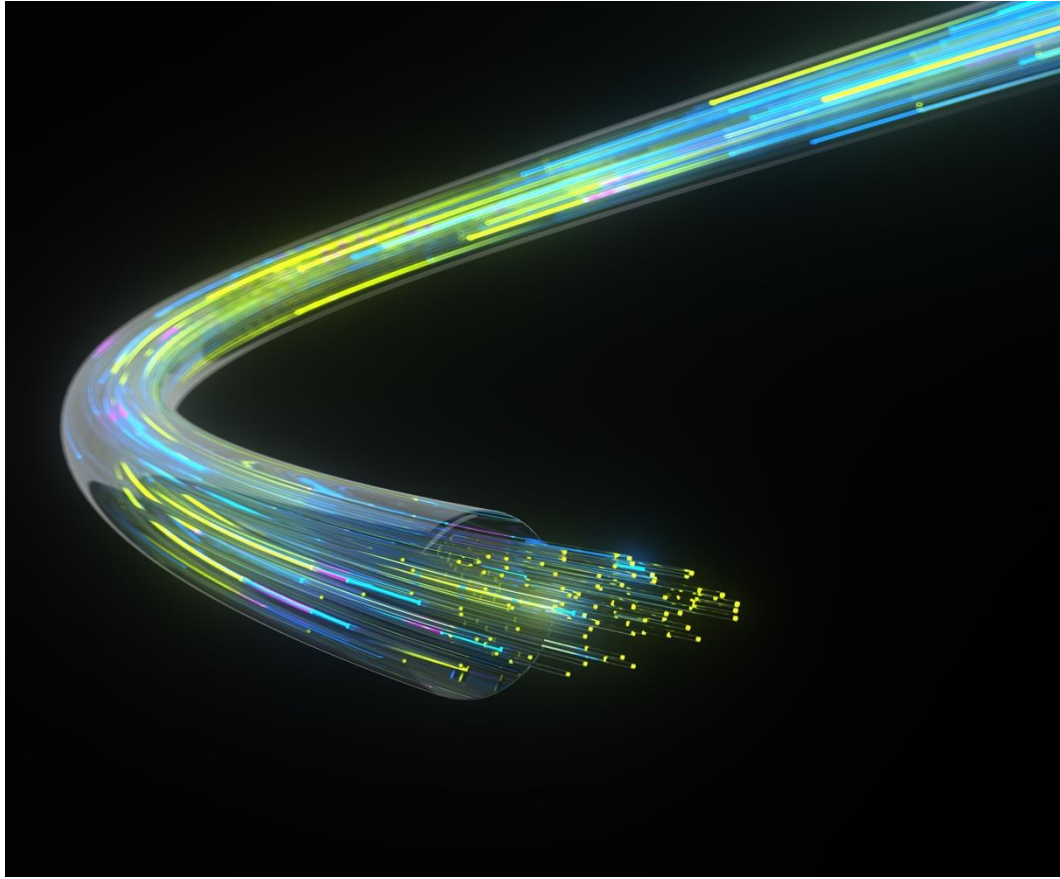
STREAMING HTTP - 3



With normal HTTP, you wait for the full newspaper to be printed, bundled, and delivered to your door. You get everything at once — but you have to wait until it's completely done before you read a single word.

With streaming HTTP, a reporter at a live event texts you sentence by sentence as things happen. You're reading the story in real time while they're still writing it. When they send "that's all" — the stream is done.

STDIO VERSUS HTTP



stdio only works locally — both the client and server must be on the same machine. HTTP works over a network — your server can run on a remote host, a container, or in the cloud HTTP supports multiple clients simultaneously — stdio is a one-to-one pipe.

HTTP works behind load balancers, through proxies, and across organizational boundaries — solving the deployment pain that keeps stdio servers trapped on localhost.

STREAMING HTTP - EXAMPLE

```
# Server.py
# Create the server instance, specifying host and port
mcp = FastMCP(
    name="my-http-server",
    host="127.0.0.1",
    port=8765,
)
...
```

MCP.JSON

```
{
  "servers": {
    "myHttpServer": {
      "type": "http",
      "url": "http://127.0.0.1:8765/mcp"
    }
  }
}
```

LOGGING

MCP servers have two separate logging channels and students need to understand both:

- Channel 1 — Server-Side Logging (Python's built-in logging module)
This writes to a file or the terminal on the *server machine*. Nobody outside the server sees it. It's for the developer — debugging, monitoring, audit trails stored on disk.
- Channel 2 — Client-Side Logging (ctx methods)
This sends log messages *over the MCP protocol* to the connected client (e.g., Claude Desktop, VS Code Copilot Agent). The client receives them in real time. It's for showing progress and status to whoever is using the tool.

Both channels work at the same time and serve different audiences.

SERVER-SIDE LOGGING

Step 1 — Configure logging with `basicConfig`

`basicConfig` is a one-time setup call at the top of your server file, before any tools are defined. It tells Python's logging system where to send messages, what format to use, and what minimum severity level to capture. You pass it a level (such as `INFO`), a format string controlling what each log line looks like, and optionally a filename if you want logs written to disk instead of the terminal.

Step 2 — Create a logger with `getLogger`

After `basicConfig`, you create a logger object by calling `logging.getLogger(__name__)`. Think of it as getting your own named pen to write entries in a shared log book. The `__name__` argument automatically uses the current module's name, making it easy to trace which part of your code produced a given entry. Assign the result to a variable named `logger` and use it throughout your file.

SERVER-SIDE LOGGING - 2

Step 3 — Write log messages with `logger.info`

Call `logger.info("your message here")` anywhere something worth recording happens — a tool being called, a result returned, or an error caught. The `info` level is for normal, expected operations. For other situations use `logger.error()`, `logger.warning()`, or `logger.debug()` for noisy low-level detail.

FOUR LOG LEVELS

Level	When to use it
DEBUG	Noisy detail — input values, internal steps. Off in production.
INFO	Normal operations — tool called, task started, task done.
WARNING	Something odd happened but the tool still worked.
ERROR	The tool failed. Log what went wrong and why.

SERVER-SIDE LOGGING - EXAMPLE

```
import logging
from fastmcp import FastMCP, Context

# Step 1 - configure logging once at the top
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s %(levelname)s %(message)s",
    filename="audit.log"
)

# Step 2 - create a named logger for this module
logger = logging.getLogger(__name__)

mcp = FastMCP("Customer Server")
```

SERVER-SIDE LOGGING – EXAMPLE - 2

```
≡ audit.log ×
```

```
≡ audit.log
```

```
1 2026-06-19 09:27:59,536 INFO Starting worker 'DonisLaptopBig#10052' with the foll
2 2026-06-19 09:27:59,537 INFO * trace(message: str, ...)
3 2026-06-19 09:27:59,537 INFO * fail(message: str, ...)
4 2026-06-19 09:27:59,537 INFO * sleep(seconds: float, ...)
5 2026-06-19 09:27:59,558 INFO Processing request of type ListPromptsRequest
6 2026-06-19 09:27:59,559 INFO Processing request of type ListToolsRequest
7 2026-06-19 09:29:01,696 INFO Processing request of type CallToolRequest
8 2026-06-19 09:29:01,699 INFO TOOL_START | get_customer | customer_id=12
9 2026-06-19 09:29:01,707 INFO TOOL_SUCCESS | get_customer | customer_id=12
10
```

CLIENT-SIDE LOGGING



Client-side logging is how your MCP tool sends live status messages to whoever is using it — the MCP client app such as Claude Desktop or VS Code Copilot Agent.

While server-side logging writes silently to a file on disk, client-side logging puts real-time feedback directly in front of the user as the tool runs.

It uses the `ctx` object (short for Context) that FastMCP passes into every tool automatically.

CLIENT-SIDE LOGGING - 2

Step 1 — Add ctx: Context to your tool's parameters

Every FastMCP tool can receive a special ctx parameter by declaring it in the function signature. You import Context from fastmcp and add ctx: Context as a parameter. FastMCP injects it automatically — you never pass it yourself when calling the tool. Once you have it, you have access to all the client-side logging methods.

Step 2 — Call the right log level method

The ctx object has four logging methods matching standard severity levels. Call `await ctx.debug()` for low-level detail, `await ctx.info()` for normal progress updates, `await ctx.warning()` for something unexpected but non-fatal, and `await ctx.error()` for failures. Each one sends a live notification over the MCP connection to the client.

CLIENT-SIDE LOGGING - 3

Step 3 — Place log calls at key moments in your tool

Log at the start so the client knows the tool is running, log during any major step, and log at the end to confirm completion or report failure. Keep messages short and plain — the client displays them as status updates, not technical output.

CLIENT-SIDE LOGGING - 4

The Context object in FastMCP gives every tool a direct line back to the MCP client. When you add `ctx: Context` to a tool's parameters, FastMCP automatically injects a live connection object tied to that specific request. You then call `await ctx.info()`, `await ctx.warning()`, `await ctx.error()`, or `await ctx.debug()` at any point inside the tool to send a real-time message to the client. The client receives it immediately while the tool is still running — not after it finishes.

What Happens Behind the Scenes: Every time you call a `ctx` logging method, FastMCP does two things at once: it sends the message to the connected MCP client over the protocol, and it writes the same message to the server's internal Python logging system. You get both behaviors from one line of code. The message travels as a JSON-RPC notification over the MCP connection, carrying the severity level and your message text.

CLIENT-SIDE LOGGING - 5

```
from fastmcp import FastMCP, Context

mcp = FastMCP("Demo Server")

@mcp.tool()
async def greet(name: str, ctx: Context) -> str:
    """Greet a user by name"""
    await ctx.info(f"Starting greeting for: {name}")
    result = f"Hello, {name}!"
    await ctx.info("Done.")
    return result

if __name__ == "__main__":
    mcp.run()
```

ERROR HANDLING



In traditional applications, exceptions might result in error pages or logged messages.

In MCP applications, exceptions directly impact the AI's ability to provide helpful responses — when an LLM invokes a tool, it anticipates a result, and if that result is an unhandled exception that crashes the server, the conversation ends.

The goal isn't just "don't crash" — it's "tell the AI what went wrong so it can recover or explain it to the user."

TOOLERROR - 2



In MCP, you handle error conditions with `ToolError`.

Raising `ToolError` isn't a failure of error handling — it *is* the robust error handling. `FastMCP` catches it internally, converts it into a properly structured tool result and sends that back over the transport.

The server loop keeps running and is ready for the next call immediately. Nothing about raising `ToolError` threatens the process's ability to continue correctly.

TOOLERROR - 3

When a tool raises a regular Python exception, the LLM can't retry intelligently, and it can't tell the user what went wrong. ToolError is FastMCP's way of saying this error is intentional and this message should go to the client.

```
from fastmcp import FastMCP, ToolError

@mcp.tool()
def divide(a: float, b: float) -> float:
    """Divide two numbers."""
    if b == 0:
        raise ToolError("Cannot divide by zero")
    return a / b
```

EXCEPTION

A broad catch-all is necessary on occasion so unexpected errors don't crash the server, but it's also the easiest place to accidentally leak sensitive data, because you're catching exceptions you didn't anticipate and therefore can't predict the contents of. Common leak vectors:

- Database errors that include connection strings or table/column names in the message
- File-system errors that reveal internal paths
- HTTP client errors that echo back internal hostnames, ports, or auth headers

For that reason, do not display the str(e) caught on an exception straight into the ToolError message or returned in the tool result.

POP QUIZ: EXCEPTION

How should you handle a general exceptions?



10 MINUTES

ERROR HANDLING - STRATEGIES

Strategy 1 — Validate Inputs Early and Return Clear Messages

Check inputs before doing real work, and collect *all* validation problems at once rather than failing on the first one. Returning all validation errors together — rather than one at a time — gives the calling AI everything it needs to fix the request in a single retry. This avoids the exception entirely.

Strategy 2 — Catch Specific Exceptions Before General Ones

Always order exceptions from most derived to least derived, since all exceptions derive from the base Exception class — this ensures more specific exception types are handled appropriately before falling back to a generic catch-all. A typical pattern:

```
try:
    result = run_query(user_id)
except ValueError as e:
    return f"Error: {str(e)}" # known domain error
except sqlite3.Error as e:
    return f"Error: Database operation failed. {str(e)}" # known system error
except Exception as e:
    return f"Error: Unexpected system error occurred." # catch-all
```

ERROR HANDLING - STRATEGIES

Strategy 3 — Retry Transient Failures with Backoff

For transient errors, use retry mechanisms with incrementally increasing wait times between attempts, a maximum retry limit to prevent infinite loops, and randomized intervals to prevent synchronization issues. This matters for things like flaky network calls or temporarily unavailable external APIs.

Strategy 4 — Log Every Error With Context, But Sanitize It

Effective debugging requires structured logging that balances detail with security — log with context, but sanitize sensitive data before writing it. Pair this with the `ctx.error()` client-side log so both the server audit trail and the live client feedback capture the failure.

Strategy 5 — Let Middleware Handle the Boring Parts

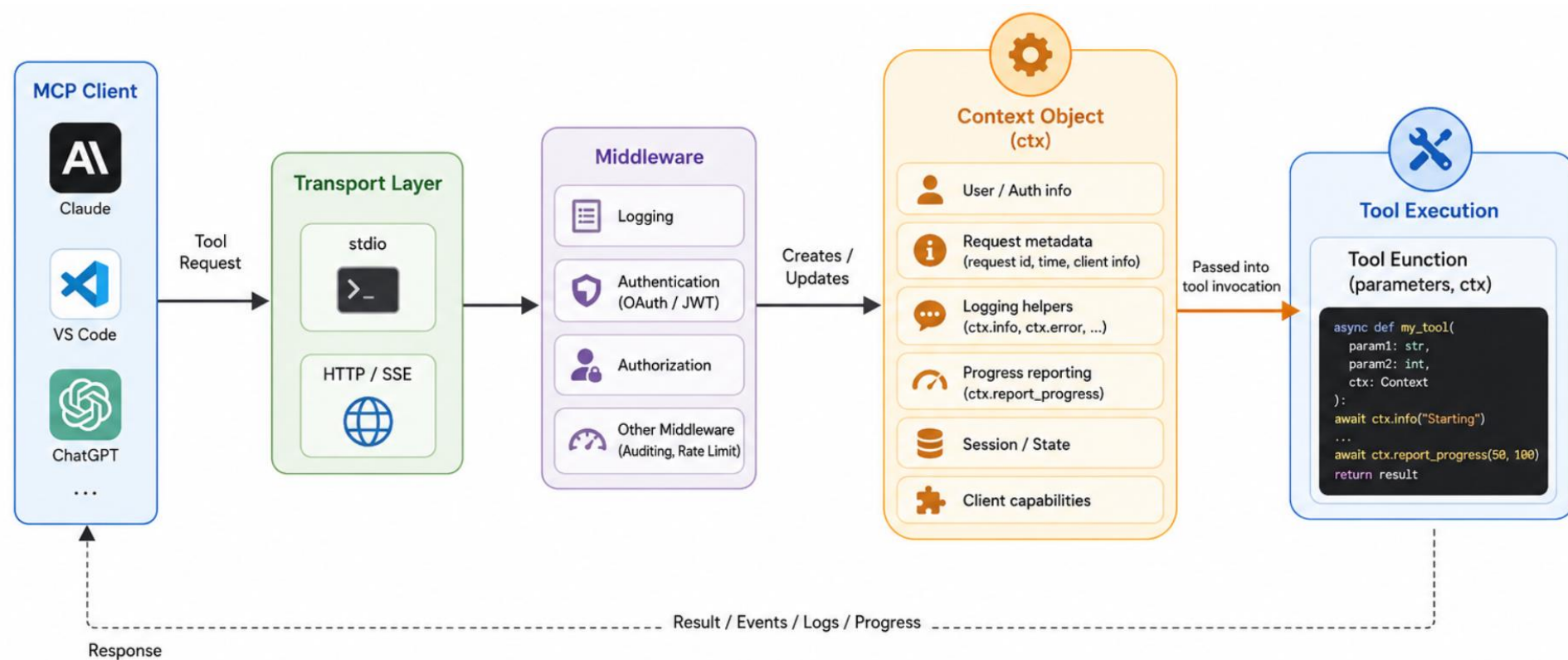
FastMCP's `ErrorHandlingMiddleware` provides consistent error response generation by catching exceptions and converting them into formal MCP `ErrorData` objects, and it maps standard Python exceptions to MCP error codes — for example, `ValueError` becomes "Invalid params." This means you don't have to manually translate every exception type into protocol-correct error responses in every single too

CONTEXT

In an MCP server, a tool normally receives only the arguments supplied by the client. For example, a calculator tool might receive two numbers to add together. Sometimes, however, a tool needs more information than just its arguments. It may need to know who called it, report progress back to the client, write log messages, or access information attached to the current request. This additional information is provided through the Context object.

You can think of the Context object as the tool's link to the MCP framework while it is running. The tool parameters tell the tool what work to perform, while the Context object provides information about the environment in which the tool is executing. Through the Context object, a tool can interact with the MCP runtime, communicate with the client, access request metadata, and retrieve information such as authentication details that were attached earlier in the request pipeline.

CONTEXT



Key Idea

The Context object is created by the MCP framework for each request and travels down the pipeline. Middleware enriches it with information (like authenticated user details). Tools use it to access request information and communicate back to the client.

What tools get

Parameters (what to do)

```
{
  "username": "jsmith",
  "repo": "my-repo"
}
```

+

Context (about the request)

```
user: donis
role: admin
request_id: 12345
session_id: abcde
...
```

CONTEXT – SERVER.PY

```
from fastmcp import Context, FastMCP

mcp = FastMCP("context-add")

@mcp.tool()
async def add(a: float, b: float, ctx: Context) -> float:
    """Add two numbers and report the result to the client via context."""
    result = a + b
    await ctx.info(f"Computed result: {a} + {b} = {result}")
    return result

if __name__ == "__main__":
    mcp.run(transport="stdio")
```


USING CONTEXT

`show_server_log` : `show_server_log` is registered on the client-side context object when it's created (`log_handler=show_server_log`), and it stays active for the whole session. On the server side, whenever the tool's code calls something like `await ctx.info(...)` or `await ctx.log(...)`, that message travels across the connection and lands in `show_server_log`, which prints it.

`show_progress`: `show_progress` is registered on the client-side context object too, but only for one specific call (`client.call_tool(..., progress_handler=show_progress)`). On the server side, whenever the tool's code calls `await ctx.report_progress(50, 100)`, that update travels across and lands in `show_progress`, which prints it.

CONTEXT – CLIENT.PY (DIFFERENT SERVER.PY)

```
async def main() -> None:
    # Open a connection to the MCP server.
    # log_handler=show_server_log means server log messages will come here.
    async with Client(str(SERVER_PATH), log_handler=show_server_log) as client:
        # Call the greet tool on the server.
        # progress_handler=show_progress means progress updates are printed live.
        result = await client.call_tool(
            "greet",
            {"name": "Username"},
            progress_handler=show_progress,
        )
        # Print the final tool result after the server finishes its work.
        print(f"result: {result.data}")
```

RETROSPECTIVE



In this module, you learned how asynchronous programming enables MCP servers to efficiently handle I/O-bound operations such as API calls, database access, and communication with external services.

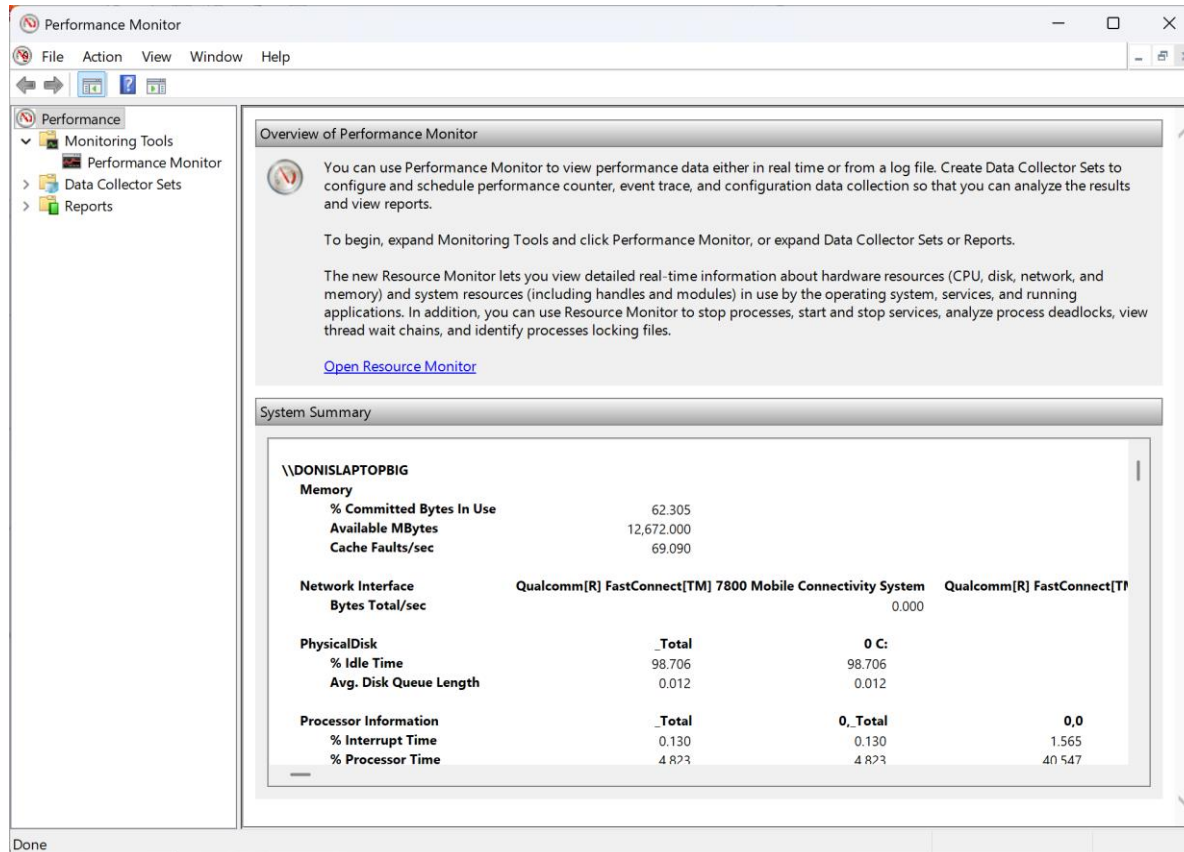
You also explored Streamable HTTP, logging strategies, error handling techniques, and the use of `ToolError` to provide meaningful feedback to AI clients while keeping servers running reliably.

Finally, you learned about the `Context` object.

LAB 3 – MCP CLIENT



GOAL



This lab walks students through finding a Windows performance counter in Performance Monitor, then querying that same counter through a custom MCP server using a natural-language prompt in GitHub Copilot Agent mode, with server-side logging added to confirm each request.

QUERYING PERFMON AND MCP

By the end of this lab, students will be able to:

- Locate a performance counter using the built-in Windows Performance Monitor
- Run a custom MCP server that exposes a perfmon counter as a resource
- Call that resource using a natural-language prompt in GitHub Copilot Agent mode
- Add basic server-side logging and confirm results on the client side

FINDING A COUNTER IN PERFMON

Step 1

- Press Win + R to open the Run dialog.
- Type perfmon and press Enter. Windows Performance Monitor opens.
- In the left panel, select Performance Monitor (under Monitoring Tools).
- Click the green + icon in the toolbar (or right-click the graph and choose Add Counters).
- In the Add Counters dialog, scroll the left list until you find the object Processor.

FINDING A COUNTER IN PERFMON

- Select the counter % Processor Time underneath it.
- In the Instances of selected object list, select _Total.
- Click Add, then OK.

You should now see a live graph of total CPU usage. This is exactly the data the MCP server will expose:

- Object → Processor(_Total)
- Counter → % Processor Time

OPEN THE WORKSPACE

Steps

- Navigate to the Lab3 folder provided for this class.
- Open the performance subfolder in VS Code as your workspace (File...Open Folder).
- Confirm you see perfmon_server.py in the Explorer panel.

PERFMON MCP SERVER

- Within the MCP.json configuration file, press the Start link to launch the MCP server.
- Other than the configuration file, confirm that the perfmon server is running.
- Open the Copilot Chat panel and switch the mode dropdown to Agent.
- Enter the following prompt in the chat box:
- Get the performance counter for: "% Processor Time" on the object "Processor(_Total)".
- Confirm Copilot returns a CPU usage value in the chat pane.

SERVER-SIDE LOGGING

The server side is where we plan to add file-based logging. Use the sample files presented earlier as a template to complete these steps. Name the log file: `perf_server.log`.

Steps:

- Open `perfmon_server.py`.
- Add an import for Python's built-in logging module.
- Configure logging to write to a file named `perf_server.log`, at the INFO level, with each entry including a timestamp.
- Inside the resource function that handles counter requests, add a log entry as the very first line — before any other processing happens.
- Have that log entry record that a request was asked, along with the object name and counter name that were requested.
- Save the file.

REVIEW LOG FILE

Steps

- In the Copilot Chat panel, re-run the prompt from earlier in the lab to display the object / counter.
- Confirm the counter value is displayed in the chat pane (this is your client-side confirmation — no log file needed).
- Open perf_server.log and confirm a line showing the request start time appears (this is your server-side confirmation).

BONUS

Try other counters with the MCP server:

- Object: Memory — Counter: Available MBytes
- Object: PhysicalDisk(_Total) — Counter: % Disk Time
- Object: System — Counter: Processor Queue Length
- Object: LogicalDisk(C:) — Counter: % Free Space
- Object: Memory — Counter: Pages/sec

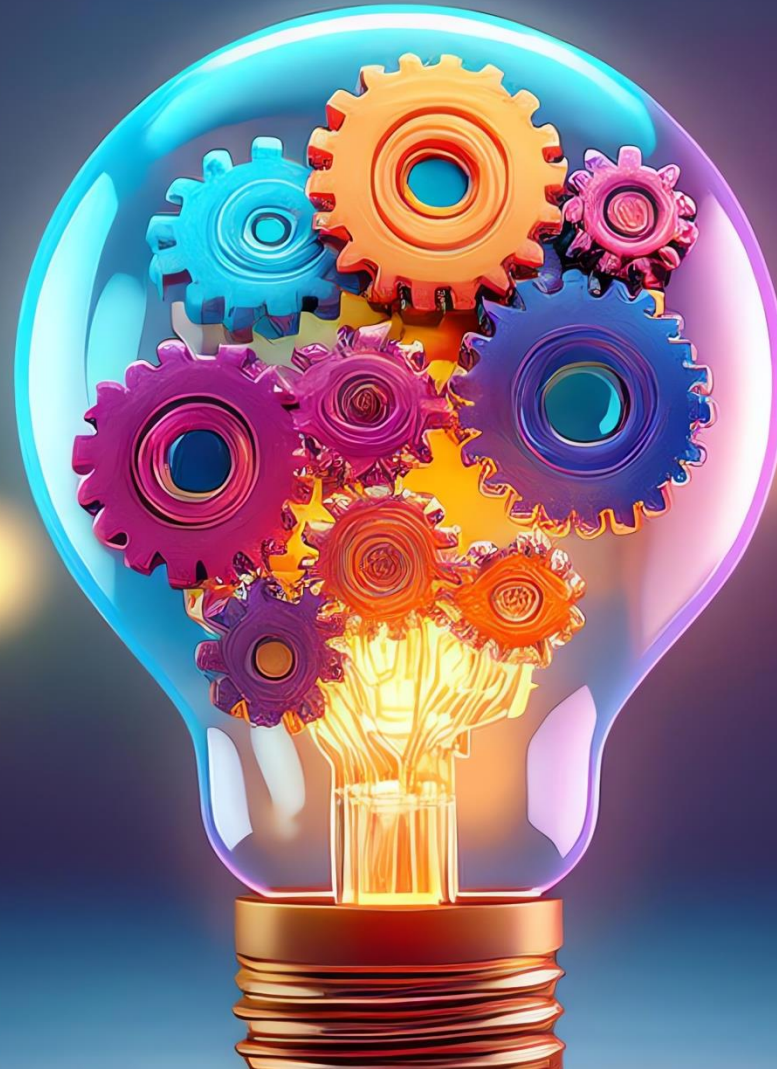
Lab completed

“Develop a passion for learning.”



SECURITY

JWT and OAUTH2



2

INTRODUCTION



In this module, you will learn the security concepts needed to build secure MCP servers. Topics include security by design, least privilege, JWT bearer tokens, middleware-based authentication and authorization, and OAuth 2.0. The goal is to protect both the MCP server and the external systems it accesses.

You will explore how FastMCP supports secure architectures through middleware, context propagation, and token validation. By the end of the module, you will understand how to authenticate users, enforce permissions, and securely connect MCP servers to external services.

CONTEXT



MCP servers can access sensitive data and systems — security isn't optional, it's foundational.

MCP servers are not just code — they are active agents that can read files, query databases, call external APIs, and take actions on behalf of users. That level of access makes them a high-value target. A poorly secured MCP server isn't just a bug, it's an open door to your entire system.

FIRST-CLASS CONCERN

Security needs to be a first-class concern from the moment you start designing your server. The decisions you make early — how tools are structured, what they can access, how requests are handled — will either make your server defensible or leave it exposed. Retrofitting security onto an insecure foundation is painful and often incomplete.

Think of security as a series of layers, not a single lock:

- Who is allowed to connect to this server?
- What are they allowed to do once connected?
- What happens when something goes wrong?
- How will you know if something has been compromised?

POP QUIZ: THIS IS CALLED WHAT?

Start by mapping every tool in your server to the data and systems it touches, then ask whether each connection is necessary.



5 MINUTES

DESIGN

"Security by design" means building access controls and privacy protections in from the start, not bolting them on later.

It's a philosophy, not a checklist. Every architectural decision — how data flows, where it's stored, who can call what — should prompt the question: "what are the security implications?" This shifts security from a final review step to an ongoing design constraint.

Privacy by design applies the same thinking to personal data. If your MCP server handles user information, decide upfront what's actually necessary to collect, how long to retain it, and who can access it. Collecting more than you need isn't just wasteful — it's a liability.

POP QUIZ: SECURITY BY DESIGN

Security by design is an example of what security concept?



5 MINUTES

LEAST PRIVILEGE

The principle of least privilege means every component — a tool, a user, a service — should have the minimum access required to do its job, nothing more.

This limits the scope of the problem when things go wrong. A tool that can only read one table can't delete your database. A read-only user can't exfiltrate data through a write operation.

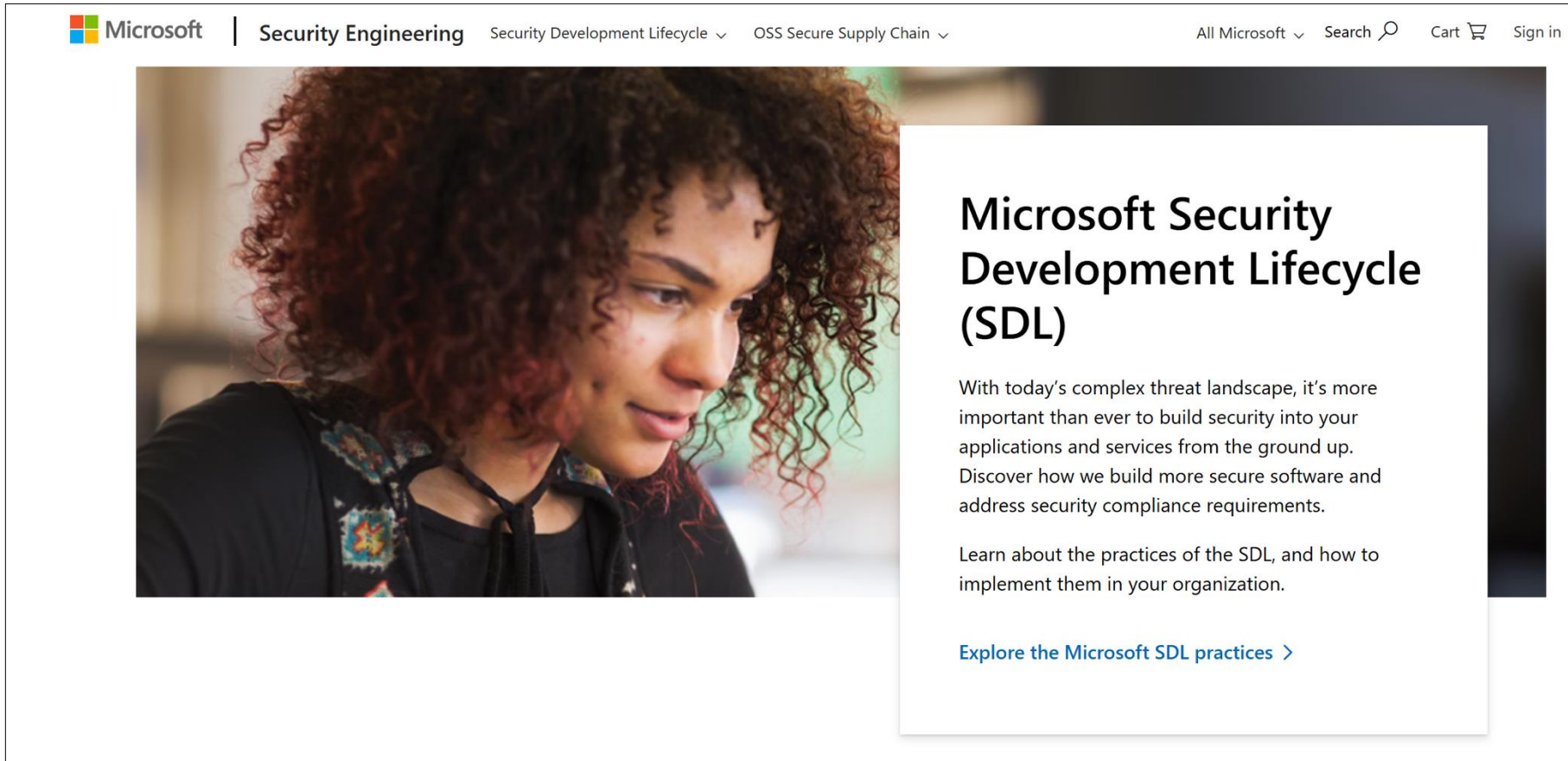
Best practices:

- Audit each tool for the minimum access it actually needs
- Use scoped API keys restricted to specific resources
- Assign database users only the permissions their queries require
- Revisit permissions regularly — access creep happens over time

MICROSOFT SDL

The Microsoft Security Development Lifecycle (SDL) is a development process that builds security into every phase of software development.

<https://www.microsoft.com/en-us/securityengineering/sdl>



The screenshot shows the Microsoft Security Engineering website. The header includes the Microsoft logo, "Security Engineering", and navigation links for "Security Development Lifecycle" and "OSS Secure Supply Chain". On the right, there are links for "All Microsoft", a search icon, a cart icon, and a "Sign in" link. The main content area features a large image of a woman with curly hair on the left. To her right, a white box contains the title "Microsoft Security Development Lifecycle (SDL)" in bold. Below the title, the text reads: "With today's complex threat landscape, it's more important than ever to build security into your applications and services from the ground up. Discover how we build more secure software and address security compliance requirements." Further down, it says: "Learn about the practices of the SDL, and how to implement them in your organization." At the bottom of the white box is a blue link that says "Explore the Microsoft SDL practices >".

JWT BEARER TOKENS

A JSON Web Token (JWT) is a compact, self-contained string used to securely transmit information between parties. When used as a "Bearer" token, it means: *whoever holds it gets access* — like a movie ticket. No need for the server to look up a session. The token itself proves who you are.

The token has three Base64-encoded sections separated by dots:

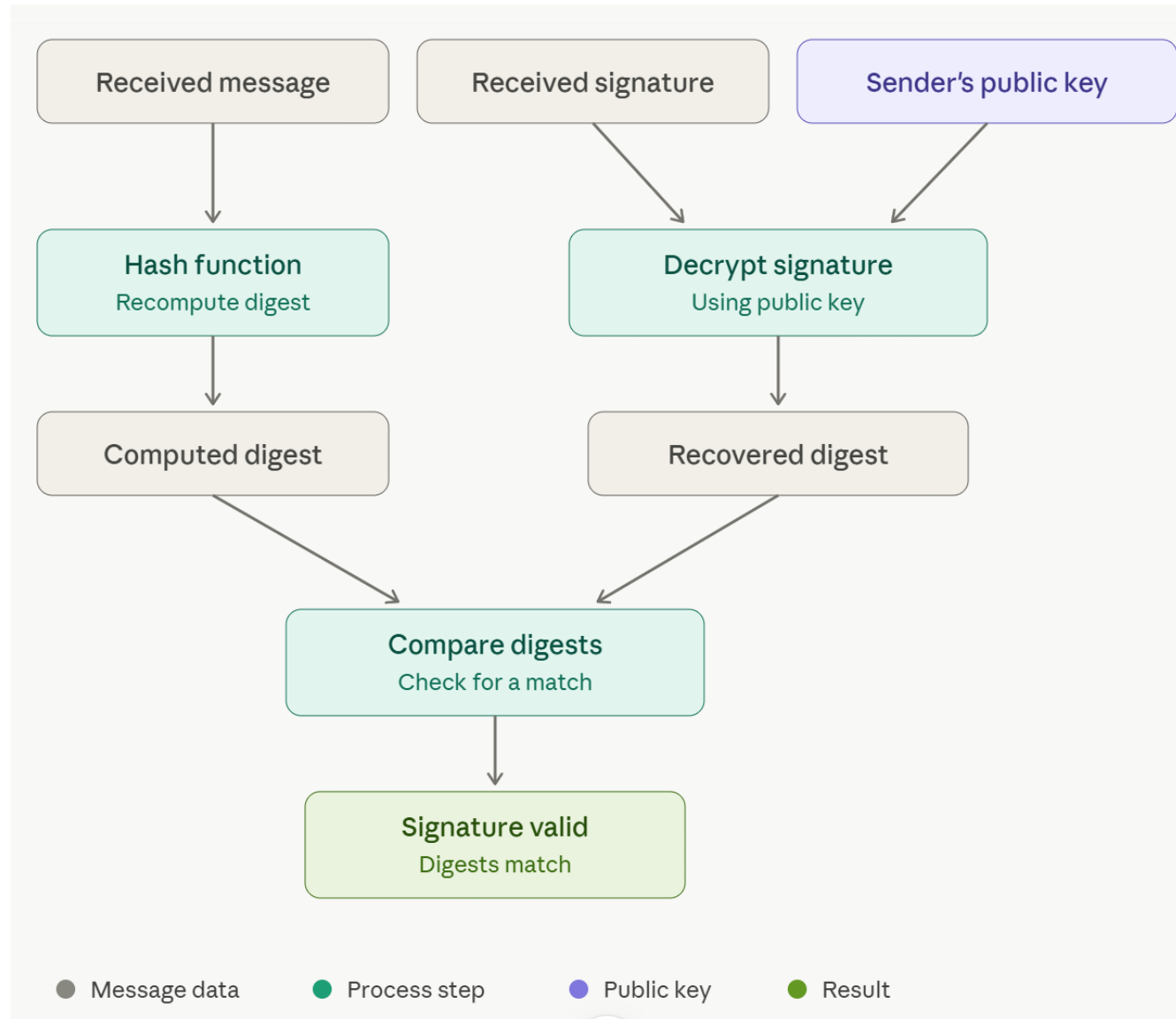
- Header — declares the token type (JWT) and the signing algorithm (e.g. HS256)
- Payload — contains the claims: user ID, roles, expiration, and any custom data
- Signature — a cryptographic hash that proves the token hasn't been tampered with

JWT BEARER TOKENS

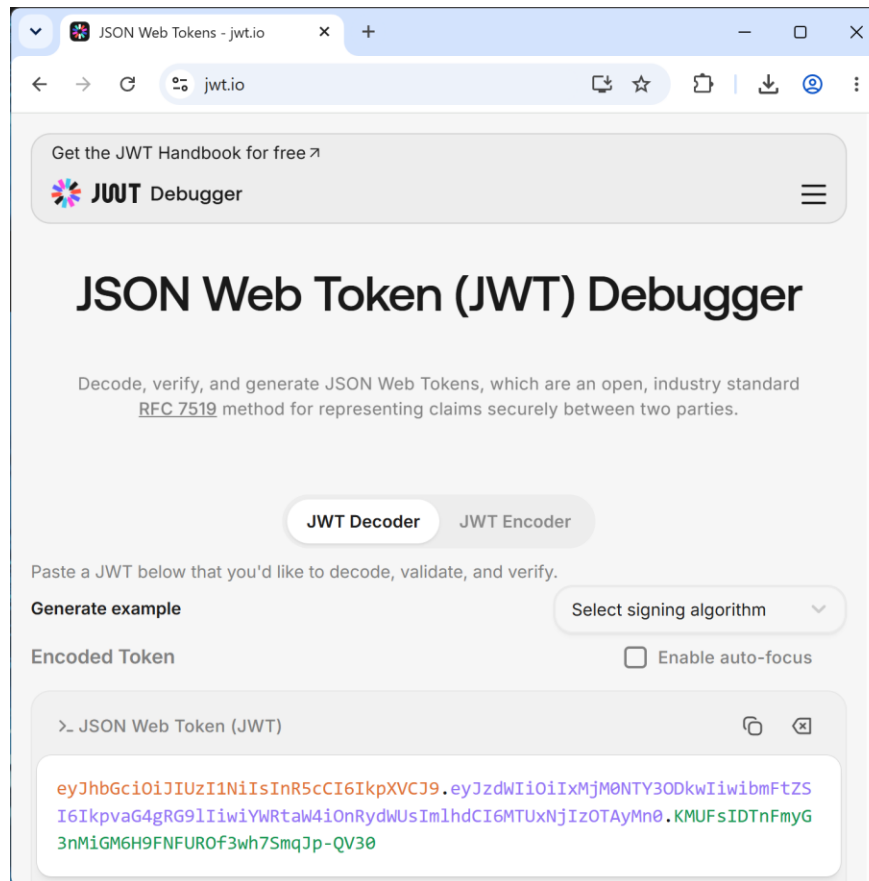
JWT tokens serve two related but distinct purposes. For authentication, a token proves who you are—when a user logs in, the server issues a signed JWT containing their identity (like a user ID), and the client sends that token with every subsequent request. The server verifies the signature to confirm the token is genuine and hasn't been tampered with, without needing to look up a session in a database every time.

For authorization, the same token can carry claims about what that user is allowed to do—like their role ("admin", "read-only") or specific permissions ("can_write", "can_delete"). When a request comes in, the server doesn't just check who you are, it also reads those claims to decide whether you're allowed to do what you're asking.

DIGITAL SIGNATURES



JWT.IO

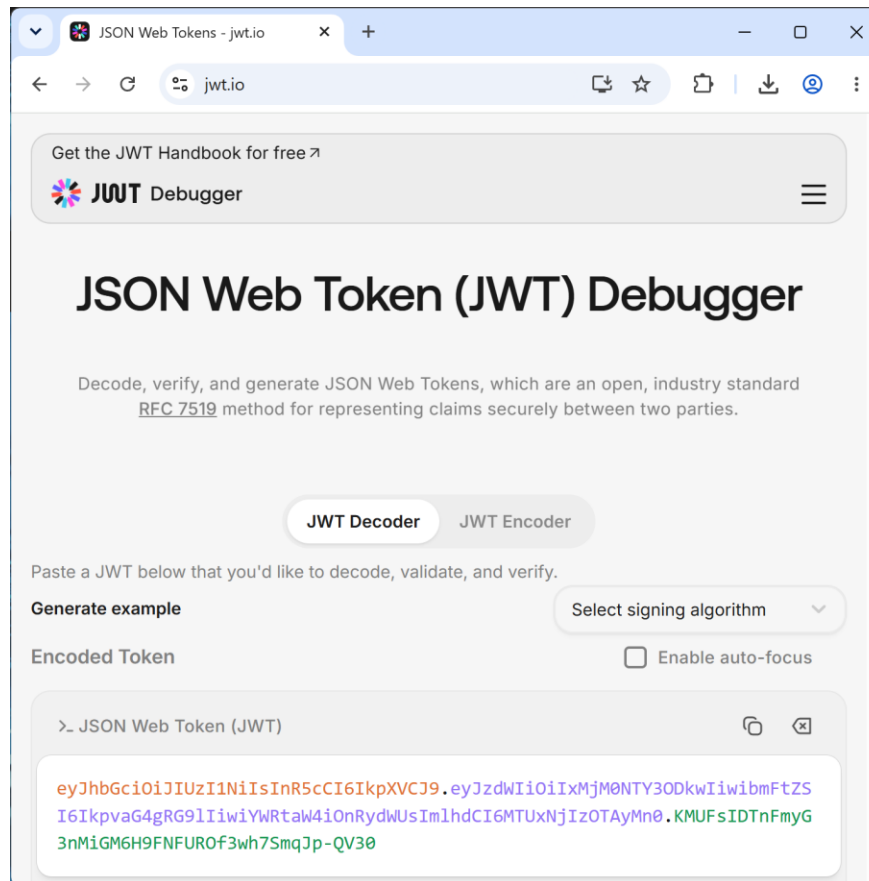


JWT.io is a tool for managing tokens.

Paste any JWT and it instantly decodes the header, payload, and signature into readable JSON.

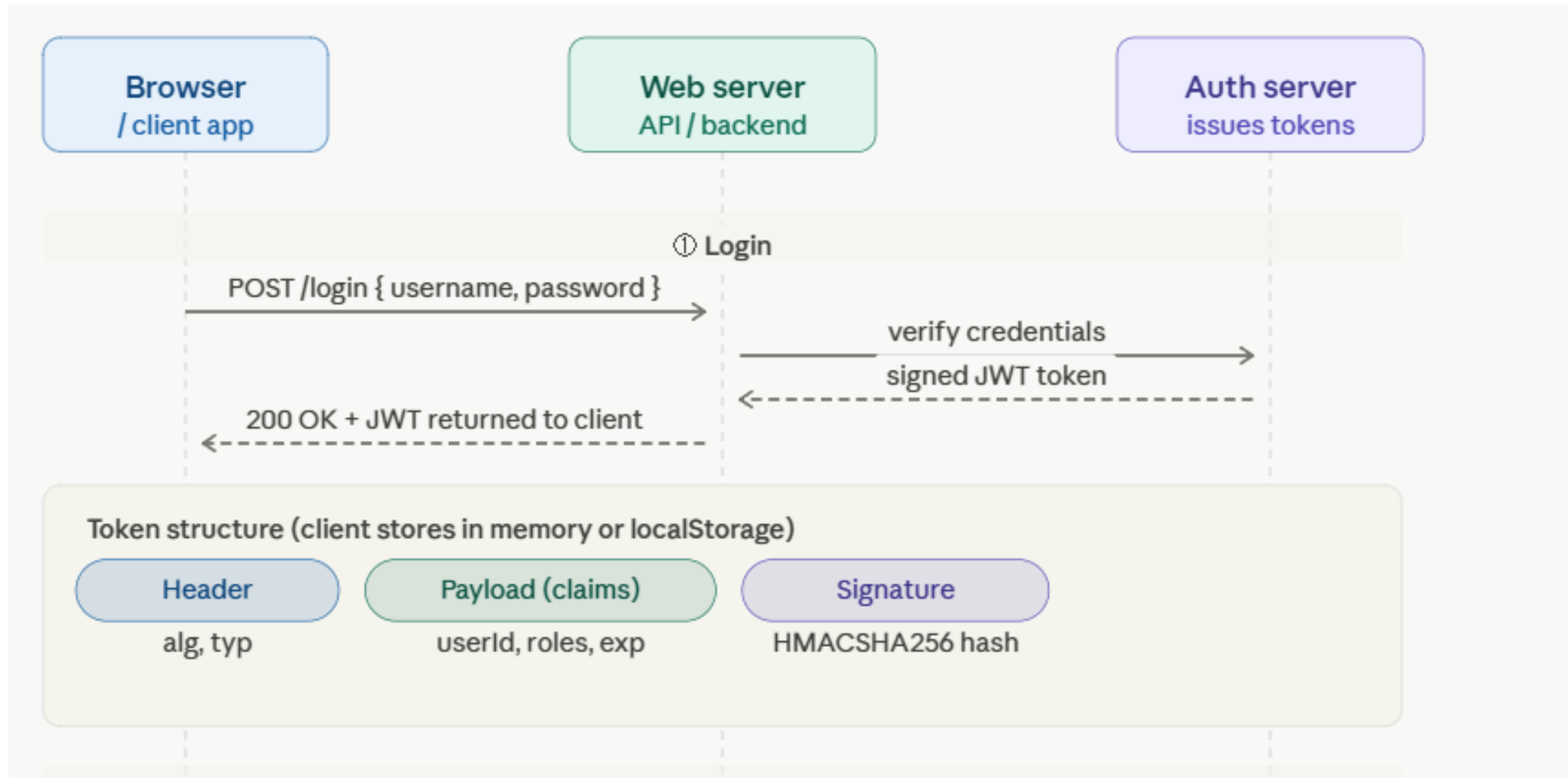
You can also build and sign your own tokens right in the browser. It's the fastest way to understand what's actually inside a token — no code required.

JWT.IO



Never put sensitive data in the payload — it's encoded, not encrypted. Anyone can decode it.

JWT WORKFLOW



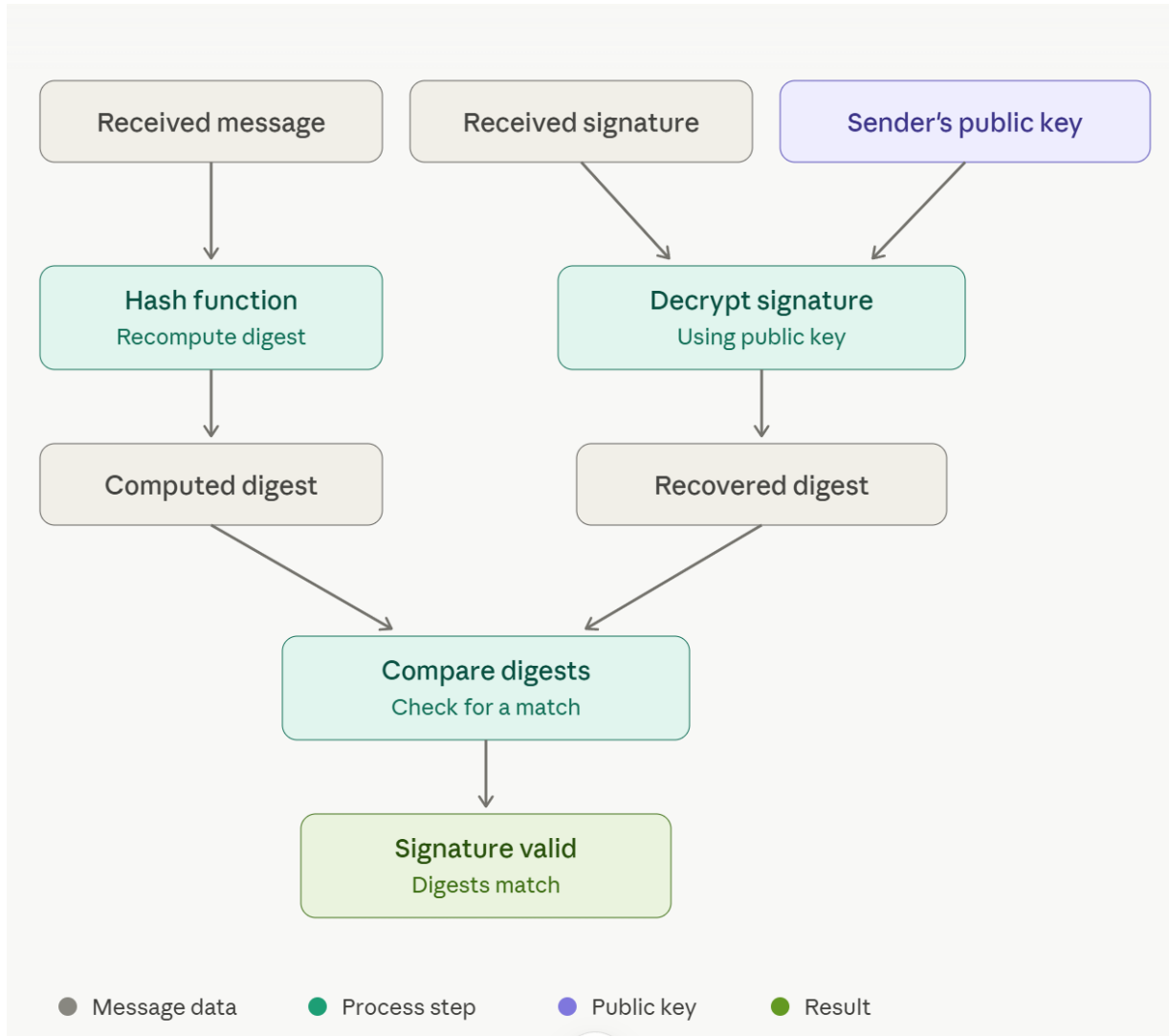
POP QUIZ: TOKEN VERIFICATION

How is the JWT token verified?



5 MINUTES

VERIFYING SIGNATURES



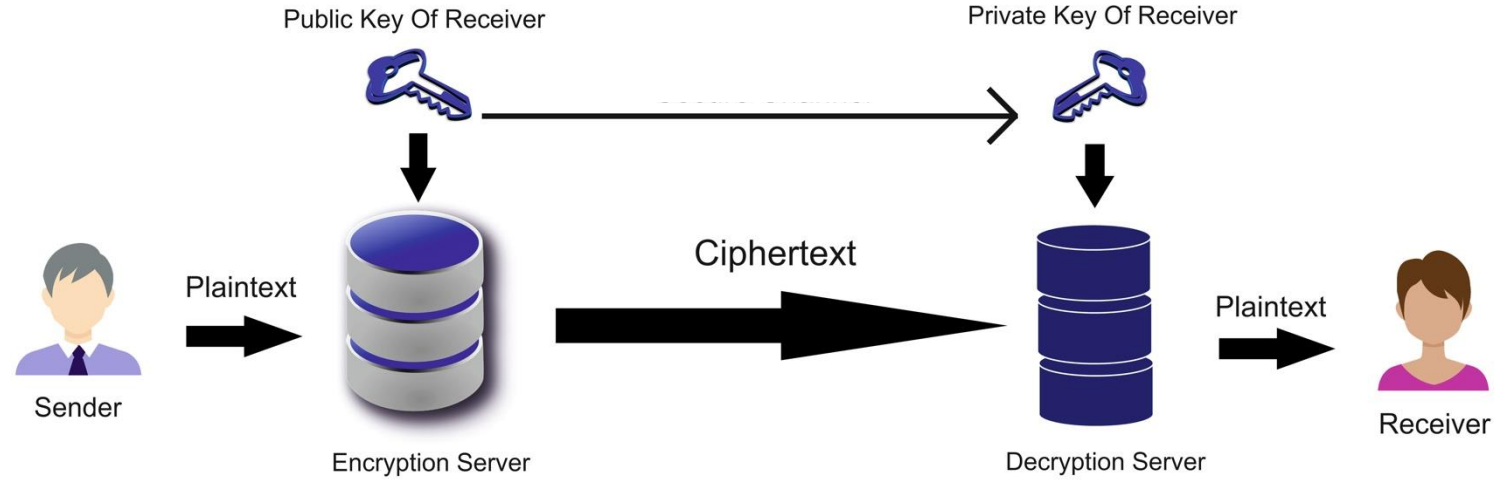
SYMMETRIC VERSUS ASYMMETRIC

The default for JWT.io is HS256.

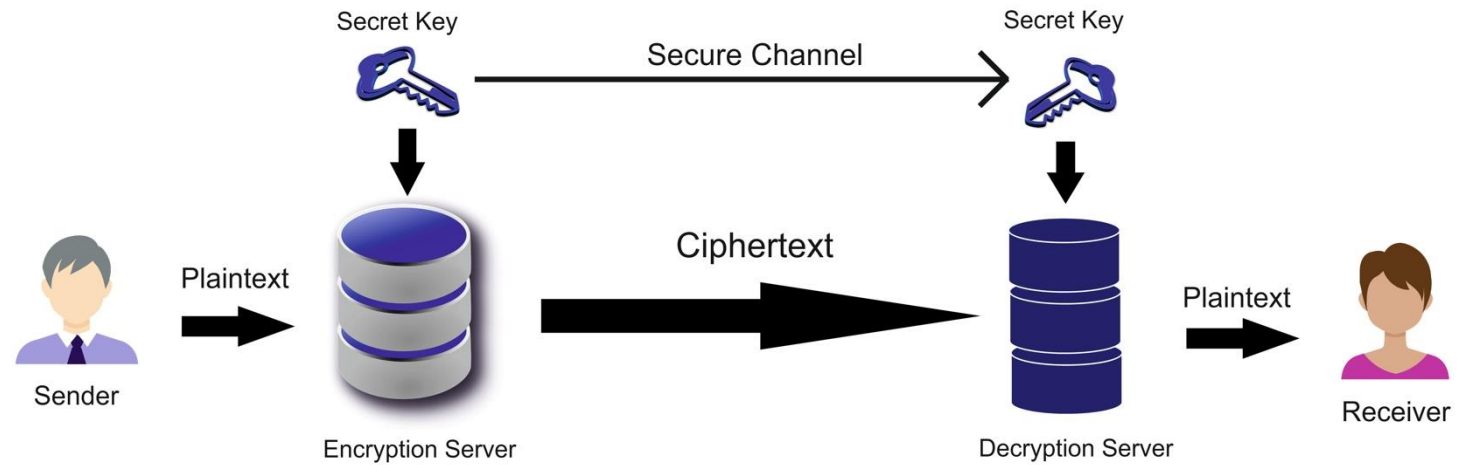
HS256, which stands for HMAC-SHA256 — the shared secret approach.

alg: "HS256" — tells the verifier which algorithm was used to create the signature. HS256 means both the signing and verification use the same secret key (symmetric). There is no public/private key pair here — whoever verifies the token must already have the same secret the Auth server used to sign it.

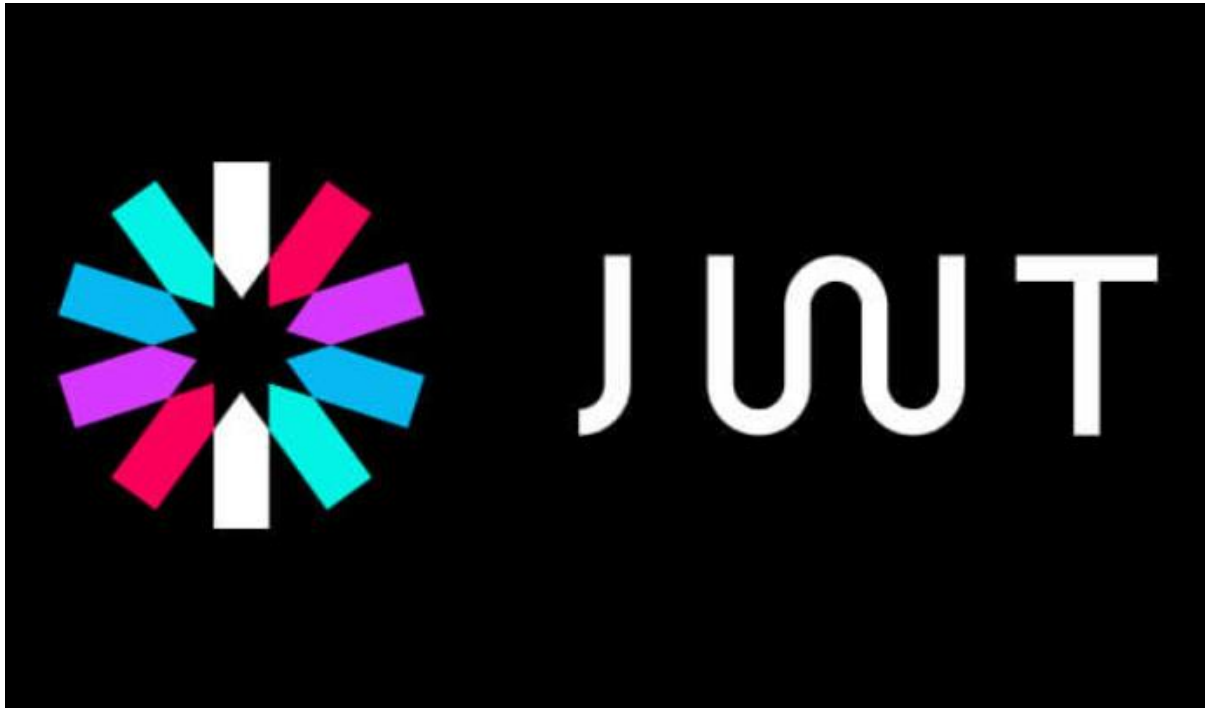
Asymmetric Encryption



Symmetric Encryption



JWT LIBRARY



The JWT library is a lightweight Python package for creating and verifying JSON Web Tokens.

You generate a token by passing in a payload dictionary and a secret key, and verify it the same way. The library handles the base64 encoding, signature, and expiration checking under the hood so you don't have to write any of that yourself.

Pass in your claims (user ID, role, expiration time, etc.) and get back a compact signed token string.

JWT BEARER TOKENS - 2

`jwt.encode()` takes a payload dictionary (your claims), a secret key, and a signing algorithm (typically HS256), and returns a compact signed token string ready to send to a client.

`jwt.decode()` does the reverse—it takes that token string plus the same secret key and algorithm, verifies the signature, and returns the original payload dictionary as plain Python data. If the token has been tampered with or expired, it raises an error. The returned payload is where your claims live—so you can read `payload["user_id"]` or `payload["role"]` directly.

JWT BEARER TOKENS

Best practices for API tokens:

- Generate tokens with a cryptographically secure random generator
- Use long tokens (at least 32 characters) to make brute-forcing impractical
- Transmit tokens only over HTTPS — never over plain HTTP
- Scope tokens to specific permissions wherever possible
- Treat tokens like passwords: never share, never log, never hardcode

JWT TOKEN – MSP.JSON

```
{
  "servers": {
    "jwt-demo-server": {
      "type": "stdio",
      "command": "python",
      "args": [
        "src/server.py"
      ],
      "env": {
        "JWT_SECRET": "${input:jwt_secret}",
        "JWT_TOKEN": "${input:jwt_token}"
      }
    }
  },
```

```
    "inputs": [
      {
        "id": "jwt_secret",
        "type": "promptString",
        "description": "Enter the JWT secret key",
        "password": true
      },
      {
        "id": "jwt_token",
        "type": "promptString",
        "description": "Enter the JWT token",
        "password": true
      }
    ]
  }
}
```


ENV AND INPUTS BLOCK

In an mcp.json file, the env block defines environment variables passed into the MCP server process at startup—things like API keys, database strings, or secret tokens. Each entry is a key-value pair the server reads at runtime, keeping secrets out of your source code and making it easy to swap values between environments.

The inputs block lets you declare values supplied at connection time—like a username or token that varies per user or session. Instead of hardcoding those values, you declare them as named inputs and reference them elsewhere in the config. The host application (like VS Code) then prompts for or injects those values when the server launches, making the same config file reusable across different users or contexts.

SERVER.PY – MAIN CODE

```
def validate_token():
    try:
        payload = jwt.decode(
            TOKEN,
            SECRET_KEY,
            algorithms=["HS256"]
        )
        return payload
    except Exception as ex:
        return None
```

```
@mcp.tool()
def delete_user(username: str) -> str:

    claims = validate_token()

    if not claims:
        return "Access Denied: Token invalid"

    if claims["role"] != "admin":
        return "Not Authorized: ..."

    return f"Deleted user: {username}
(requested by {claims['user']})"
```

MIDDLEWARE



FastMCP supports middleware when running in HTTP mode. Implement authentication as middleware that validates the token on every incoming request and attaches the verified identity to the request context.

Then implement authorization using a helper function that checks whether the attached identity has permission to invoke that tool.

REQUIRED MODULES

Middleware

Base class you subclass to create your own middleware—code that runs automatically before or after every request for the MCP server

MiddlewareContext

The object passed into your middleware containing everything about the current request—the tool being called, the incoming message, and other metadata

- `get_context`: A dependency function used inside your tool or resource handlers to grab the full FastMCP context object without passing it around manually
- `get_http_headers`: A dependency function that pulls the raw HTTP headers from the incoming request directly into your handler

SERVER.PY – MAIN CODE

```
mcp = FastMCP(  
    "secure-server",  
    middleware=[RoleMiddleware()  
)
```

This code creates a new FastMCP server with two things: a name ("secure-server") and a list of middleware to run on every incoming request.

The `middleware=[RoleMiddleware()]` part is like hiring a bouncer—every request gets checked by `RoleMiddleware` before it reaches any of your actual tools.

You could stack multiple bouncers by adding more to the list, and each would inspect the request in order before passing it along.

MIDDLEWARE EXAMPLE

```
def get_bearer_token() -> str | None:
    headers = get_http_headers(include_all=True)
    auth_header = headers.get("authorization") or headers.get("Authorization")
    if not auth_header:
        return None

    scheme, _, token = auth_header.partition(" ")
    if scheme.lower() != "bearer" or not token:
        return None
    return token
```

MIDDLEWARE EXAMPLE - 2

```
async def on_call_tool(self, context: MiddlewareContext, call_next):
    token = get_bearer_token()

    if token is None:
        raise PermissionError("Missing JWT token")

    # For a real app, verify issuer, audience, expiration, etc.
    claims = jwt.decode(
        token,
        SECRET_KEY,
        algorithms=["HS256"]
    )
```

```
if context.fastmcp_context is None:
    raise PermissionError("Missing request context")

context.fastmcp_context.set_state("user", {
    "id": claims.get("sub"),
    "role": claims.get("role")
})

return await call_next(context)
```


MIDDLEWARE EXAMPLE - 3

```
@mcp.tool()
async def admin_only_message() -> str:
    user = get_context().get_state("user")
    if not user:
        raise PermissionError("Missing user context")

    if user.get("role") != "admin":
        raise PermissionError("Access denied. Admin role required.")

    return f"Hello {user.get('id')}. You are an admin."
```

MIDDLEWARE – CLIENT.PY

```
def create_jwt():  
    return jwt.encode(  
        {  
            "sub": "donis",  
            "role": "admin"  
        },  
        SECRET_KEY,  
        algorithm="HS256"  
    )
```

```
async def main():  
    token = create_jwt()  
  
    client = Client(  
        "http://127.0.0.1:8000/mcp",  
        auth=token,  
    )  
  
    async with client:  
        result = await client.call_tool("admin_only_message", {})  
        print(result)
```

OAUTH

An MCP server acts as an intermediary between an AI client and external systems.

If the MCP server only provides access to public information, such as weather data, public documentation, or local files, no user identity is required. The server can simply retrieve the information and return it to the AI.

However, when the MCP server needs to access private resources such as email, calendars, GitHub repositories, CRM systems, or bank accounts, it must know which user it is acting for and what permissions that user has granted.



OAuth - 2

OAuth 2.0 solves this problem. Instead of giving the MCP server your username and password, you authenticate directly with the external service (such as GitHub or Google). The service then issues an access token that represents your identity and permissions. The MCP server uses that access token when communicating with the external system.

For example:

A user signs in to GitHub.

1. GitHub issues an OAuth access token.
2. The MCP server receives the token.
3. The AI asks the MCP server to list repositories.
4. The MCP server sends the token to GitHub.
5. GitHub verifies the token and returns only the repositories the user is authorized to access.

OAUTH - 3

The MCP server never needs to know the user's password. It only uses the access token to perform authorized operations on the user's behalf.



OAuth - 4

Suppose you create an MCP server that can read a user's Gmail messages.

Without OAuth:

AI → MCP Server → Gmail

The MCP server has no idea which Gmail account to use. With OAuth:

User signs into Google



Google gives an access token



MCP Server stores the token



AI → MCP Server → Gmail

Now the MCP server can safely access the user's Gmail account because Google has verified the user's identity and granted permission.

OAuth - 5

OAuth is commonly used when an MCP server connects to:

System	OAuth Needed?
Gmail	Yes
Google Calendar	Yes
GitHub	Usually
Microsoft 365	Yes
Salesforce	Yes
Slack	Usually
Jira Cloud	Usually
Dropbox	Yes

BEARER TOKEN

```
HEADERS = {  
    "Authorization": f"Bearer {GITHUB_TOKEN}",  
    "Accept": "application/vnd.github+json",  
}
```

This code creates the HTTP headers used by an MCP server to make authenticated requests to GitHub on behalf of a user. The Authorization header contains an OAuth access token (GITHUB_TOKEN) and sends it as a Bearer token.

This token was previously issued by GitHub after the user signed in and granted permission to the application. Instead of sending a username and password, the MCP server presents the token, and GitHub uses it to identify the user and determine what actions are allowed.

BEARER TOKEN - 2

```
HEADERS = {  
    "Authorization": f"Bearer {GITHUB_TOKEN}",  
    "Accept": "application/vnd.github+json",  
}
```

The Accept header requests GitHub's standard JSON response format. When GitHub receives the request, it validates the access token, checks the associated permissions, and either returns the requested data or denies access.

This allows the MCP server to securely access GitHub resources without ever handling the user's password.

OAUTH – VERIFYING THE ACCOUNT

In an OAuth-enabled MCP server, this tool uses the OAuth access token that was previously obtained from GitHub and stored in HEADERS. When the tool calls the GitHub /user endpoint, GitHub validates the token and determines which GitHub account authorized it. The MCP server is not authenticating the user itself; it is relying on GitHub to verify the token and identify the user.

The purpose of this tool is to confirm which GitHub account is associated with the OAuth token. If the token is valid, GitHub returns information about the authenticated user, such as their username, name, and account ID. This is often one of the first API calls made after OAuth authentication to verify that the application is connected to the expected GitHub account.

OAUTH – VERIFYING THE ACCOUNT

```
@mcp.tool()
def github_me() -> dict:
    """
    Returns the GitHub account connected to this OAuth token.
    Useful for confirming the token belongs to the expected user.
    """
    response = requests.get(
        "https://api.github.com/user",
        headers=HEADERS,
        timeout=20,
    )
    response.raise_for_status()
    return response.json()
```

OAUTH – LIST REPOS - 2

```
@mcp.tool()
def list_my_repos() -> list:
    """
    Lists repositories visible to this OAuth-authenticated GitHub account.
    Access depends on the OAuth scopes granted, such as repo or read:user.
    """
    response = requests.get(
        "https://api.github.com/user/repos",
        headers=HEADERS,
        params={"per_page": 20, "sort": "updated"},
        timeout=20,
    )
    response.raise_for_status()
    return response.json()
```

RETROSPECTIVE



In this module, you learned how to build security into an MCP server from the start using principles such as security by design and least privilege. You explored JWT bearer tokens, middleware-based authentication and authorization, secure configuration practices, and techniques for validating user access.

You also learned how OAuth 2.0 enables MCP servers to securely access user-owned resources without handling passwords directly.

LAB 4 – CHECKING



GOAL



To implement a JWT-authenticated MCP tool that enforces a maximum withdrawal amount for a checking account, encoded as a claim in the token.

SERVER.PY - PART 1

```
checking/  
├─ .vscode/  
│   └─ mcp.json  
└─ server.py
```

In VS Code, open: labs/lab4/checking directory as the workspace. Afterwards open and review the server.py source file.

The file already includes imports, FastMCP setup, SECRET_KEY, TOKEN, and the validate_token() helper. Your job is to add the missing withdraw tool below it.

IMPLEMENT THE WITHDRAW TOOL – PART 2

Important: In the following exercise, use the example code (`delete_user`) presented earlier in class as a template. It follows the same three-step pattern you need here: validate the token, read the claims, then check a condition and return a message. Adapt the logic to fit the withdrawal use case.

IMPLEMENT THE WITHDRAW TOOL – PART 2

Below `validate_token()`, implement the `withdraw` function using the pseudocode and comments below as your guide:

```
# Decorate this function as an MCP tool
def withdraw(amount: float) -> str:
    """Add a docstring describing what this tool does."""

    # Call validate_token() and store the result
    # If the result is falsy, return an access denied message

    # Read the "name" claim from the token payload
    # Read the "max" claim from the token payload
```

IMPLEMENT THE WITHDRAW TOOL – PART 2

```
# If the requested amount is greater than max:  
#     return an error message that includes both amounts  
  
# Otherwise:  
#     return a success message with the amount and account holder name
```

Note: Use `claims.get("name", "Unknown")` and `claims.get("max", 0)` to safely read each claim — the second argument is the default value if the key is missing.

JWT.IO: CREATE THE JWT TOKEN – PART 3

Go to jwt.io/#debugger-io and fill in these fields:

Algorithm	Secret (uncheck base64)
HS256	secret

In the Payload panel, enter:

```
{  
  "name": "Your Name",  
  "max": 500  
}
```

Copy the full encoded token from the Encoded panel on the left.

RUN THE MCP SERVER: PART 4

In the VS Code terminal, from the labs/lab4/checking/ directory:

```
python server.py
```

VS Code will prompt for two values — paste carefully:

- JWT secret key: secret
- JWT token: paste the token copied from jwt.io

TEST THE PROMPTS: PART 5

In GitHub Copilot Agent mode (or MCP Inspector), try each prompt separately:

- use the jwt-bank mcp server to withdraw \$250
- use the jwt-bank mcp server to withdraw \$1000

🟢 \$250 — under limit

Success message with the amount
and your name.

🔴 \$1000 — over limit

Error message showing the amount
and the \$500 maximum.

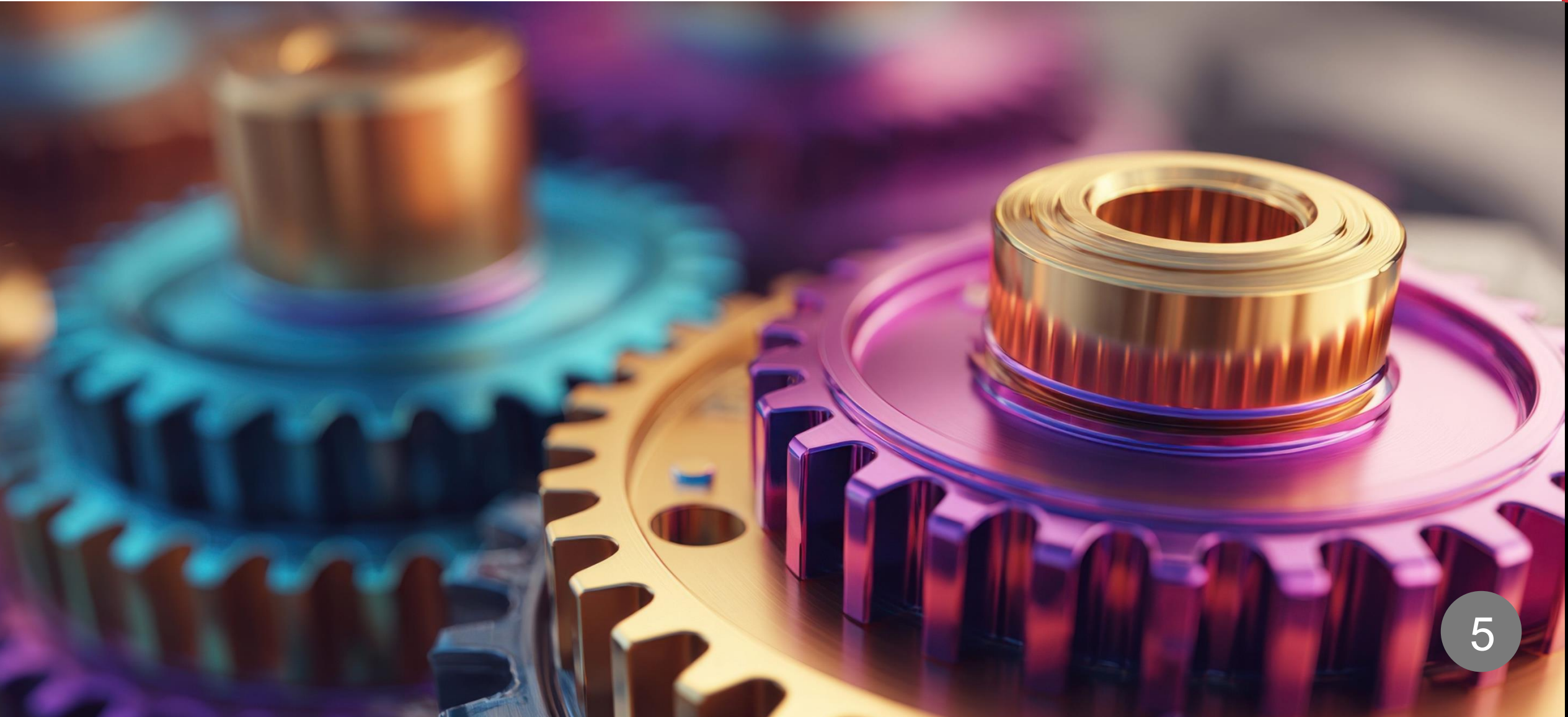
Lab completed

“Develop a passion for learning.”



NODE.JS AND MCP SERVER

DETAILS AND WORKFLOW



INTRODUCTION



In this module, you will learn how to build MCP servers using Node.js and TypeScript. The module introduces the Node.js runtime, explains why it is a strong fit for MCP development, and walks through the process of creating, configuring, and running a simple MCP server.

You will explore the MCP SDK, project scaffolding, TypeScript configuration, tool registration, and stdio transport. By the end of the module, you will understand how to create a Node.js MCP server and make it available to AI clients such as GitHub Copilot through an MCP configuration file.

NODE.JS



Node.js is a runtime environment that lets you run JavaScript outside a web browser. Node.js takes the same JavaScript engine that powers Chrome (called V8) and packages it so it can run directly on a computer or server.

This means developers can use JavaScript to build the "backend" of an application — the part that handles databases, file storage, user logins, and other behind-the-scenes logic — instead of needing a different language like Python or Java for that work.

ADVANTAGES WITH MCP

Node.js makes it fast and low-friction to build an MCP server because the language, the data format, and the ecosystem all line up with what MCP needs.

Same language, less hassle. Most MCP tooling is written in JavaScript/TypeScript. If you build your MCP server in Node.js, you're using the same language the protocol's main tools were designed for — fewer surprises, more examples to copy from.

Good at juggling. Node.js is built to handle lots of things happening at once without getting stuck waiting on any one of them — like a waiter who can take five orders at once instead of standing at one table until they're done. MCP servers often need to handle several tool requests around the same time, so this fits naturally.

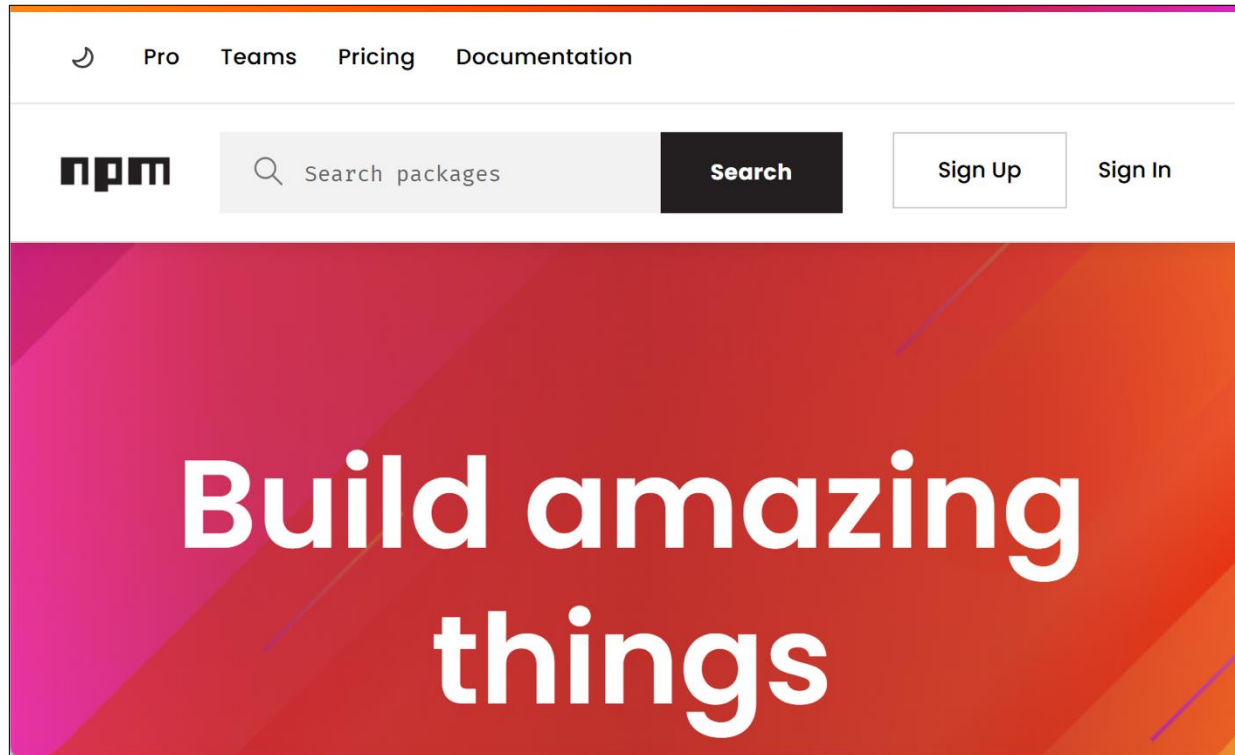
ADVANTAGES WITH MCP - 2

Speaks JSON natively. MCP messages are sent as JSON (a simple text format for structured data), and JavaScript was basically built around JSON from the start. For that reason, there's very little "translation work" needed — the data format of the protocol and the data format of the language are already a match.

Huge toolbox. Node has npm, the largest library of pre-built code packages out there. If you want your MCP server to talk to a database, call an API, or read files, there's almost always an existing package you can grab instead of writing it from scratch.

Reuse what you already have. If your company's app backend is already in Node.js (very common), you can often wrap existing functions as MCP "tools" with minimal extra work, instead of rewriting business logic in a new language just to expose it to an AI.

NPM



A public package registry containing millions of JavaScript and TypeScript packages.

The public npm registry is hosted by NPM, Inc. (owned by GitHub).

<https://www.npmjs.com/>

WALKTHROUGH – NODE.JS SOLUTION



This walkthrough demonstrates how to create a MCP server with Node.js.

SCAFFOLDING

Here is the scaffolding for a Node.js MCP server project.

- package.json lists dependencies, which *npm install* turns into node_modules
- package-lock.json is the version record
- tsconfig.json handles TypeScript settings
- src/index.ts, is where the server and its tools live
- .vscode/mcp.json, if connecting through VS Code, tells the editor where that server is and how to start it.

SCAFFOLDING - 2

```
my-mcp-server/  
├─ .vscode/  
|   └─ mcp.json  
start it  
├─ node_modules/  
hand)  
├─ src/  
|   └─ index.ts  
tool  
├─ package.json  
├─ package-lock.json  
└─ tsconfig.json
```

← tells VS Code where this server lives and how to start it

← installed packages (auto-created, never edit by hand)

← the toolbox code – your server + the get_weather tool

← project info + list of dependencies

← auto-created, locks exact dependency versions

← TypeScript settings

CREATE A NODE.JS PROJECT - STEP 1

From bash, *mkdir my-mcp-server* and then *cd my-mcp-server*

```
npm init -y
```

This just creates an empty project folder and package.json that says "this is a Node project."

INSTALL LIBRARIES – STEP 2

Line 1 — `npm install @modelcontextprotocol/sdk zod`

Installs the official MCP SDK — handles JSON-RPC messaging, tool registration, and transports — and Zod, which defines and validates the shape of your tool's input arguments.

Line 2 — `npm install -D typescript @types/node tsx`

Installs TypeScript, the Node.js type definitions, and tsx — a tool that runs TypeScript files directly without a separate compile step.

Line 3 — `npx tsc --init`

Generates a default `tsconfig.json` that tells TypeScript how to compile your code — target JavaScript version, output folder, and so on.

```
npm install @modelcontextprotocol/sdk zod
```

```
npm install -D typescript @types/node tsx
```

```
npx tsc --init
```

NEW FILES - STEP 3

In package.json, change type to, if not already present:

```
{ "type": "module" }
```

This just tells Node "use the modern way of writing code." Skip this and things break with a confusing error.

NEW FILES - STEP 3 - 2

Here is the tsconfig.json. The two settings that matter most are module and moduleResolution, both "Node16" — required because the MCP SDK is ESM-only and breaks under older settings. The rest: target picks the JavaScript version to compile to, outDir sets where compiled files go, and strict enables full type-checking to catch data mismatches before code runs:

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "Node16",
    "moduleResolution": "Node16",
    "outDir": "./dist",
    "strict": true
  }
}
```

INDEX.TS - STEP 4

Two pieces from the SDK toolbox: `McpServer` builds the server and holds your tools, `StdioServerTransport` handles communication, and using standard input/output as a private local channel — no network involved.]

`new McpServer({...})`: creates the server itself — name and version identify it to the AI app.

`server.registerTool(...)`: adds one tool, taking three things:

- the tool's internal name, a definition (title/description tell the AI when to use it)
- `inputSchema` is the Zod rule for required input)
- handler function that runs when called and returns a content array.

STEP 5

Running `npx tsx src/index.ts` starts your MCP server using stdio transport — and seeing a blank terminal afterward is expected, not an error:

- `npx` runs the tool without installing it globally
- `tsx` runs your TypeScript file directly, no compile step needed

```
npx tsx src/index.ts
```

POP QUIZ: AI CLIENT

How to make any MCP server available to an AI Assistant, such GitHub Copilot?



5 MINUTES

AI CLIENTS

Most desktop AI apps that support MCP use a config file where you add something like:

```
{
  "servers": {
    "my-mcp-server": {
      "command": "npx",
      "args": ["tsx", "/absolute/path/to/src/index.ts"]
    }
  }
}
```

This is like handing the AI app a map that says "here's where your new tool lives, and here's how to start it up."

For VS Code (with Copilot Agent mode), the same kind of entry goes in a file called `.vscode/mcp.json` inside your project.

AI CLIENTS

With mcp.json, start the MCP server.

1. Check the tool is running
2. You should see get_weather listed
3. Ask the MCP server / tool something in the GitHub copilot chat windows:
"What's the weather in {home town}?"
4. The result will be displayed in the Gitub Copilot Chat pane.



RETROSPECTIVE



In this module, you learned how Node.js and TypeScript can be used to build MCP servers. You explored project setup, npm packages, TypeScript configuration, MCP SDK components, tool registration, and stdio-based communication between an MCP server and an AI client.

You also learned how AI clients discover and launch MCP servers through configuration files such as `mcp.json`. These concepts provide the foundation for creating, testing, and integrating Node.js-based MCP servers with modern AI development tools

Module completed

“Develop a passion for learning.”



Excellent

You have completed the course successfully!



Congratulations